Universidad de Ingeniería y Tecnología · Maestría
Taller de Arquitectura Big Data · Ciclo 2026-2

Lumina Bank

Plataforma de datos batch y streaming en Google Cloud

Entrega 2: procesamiento, transformación y modelado

Bryan Montoya
David Choque
Henry Godiño

Lima, 11 de noviembre de 2026

Índice

1. Qué cambiamos después de la Entrega 1 **2**

2. Arquitectura en notación C4 **2**

2.1. Nivel 1: contexto . . . 2
2.2. Nivel 2: camino streaming . . . 3
2.3. Nivel 2: camino batch, modelado y exposición . . . 3
2.4. Conexiones entre componentes . . . 3
2.5. Capas de gobierno, seguridad y redes . . . 4

3. Por qué elegimos cada componente **7**

4. Dimensionamiento de Dataflow y Cloud Run **9**

4.1. Volumen de referencia . . . 9
4.2. Dataflow: lo que medimos . . . 9
4.3. Cloud Run . . . 11
4.4. Impacto en el costo . . . 12

5. Implementación de la Entrega 2 **12**

5.1. Pipeline batch con Spark . . . 12
5.2. Pipeline streaming con Apache Beam . . . 13
5.3. Batch y streaming en el mismo modelo . . . 13
5.4. Modelo estrella en Gold . . . 14
5.5. Modelos en BigQuery ML . . . 14
5.6. API de datos . . . 14

6. Evidencias de la implementación **14**

6.1. Proyecto e infraestructura base . . . 15
6.2. Gobierno, seguridad y redes . . . 17
6.3. Pipeline batch . . . 21
6.4. Pipeline streaming . . . 24
6.5. Pruebas de carga . . . 25
6.6. Gold y BigQuery ML . . . 28
6.7. API de datos en Cloud Run . . . 31

7. Pendientes para la Entrega 3 **33**

A. Cambios en el repositorio **34**

1 Qué cambiamos después de la Entrega 1

La revisión de la primera entrega nos dejó cuatro observaciones. Las atendimos todas y, además, desplegamos la plataforma completa en un proyecto de Google Cloud (`lumina-bank-utec`) para medir en lugar de suponer. La tabla 1 resume cada cambio.

Tabla 1. Observaciones de la docente y respuesta del equipo

Observación	Qué hicimos	Sección
Mostrar las conexiones entre componentes	Rehicimos la arquitectura en notación C4 con LikeC4: una vista de contexto, una por cada camino de datos y una por cada capa transversal. Cada flecha dice qué dato viaja y por qué canal. Numeramos 18 conexiones y las describimos en una tabla.	2
Incluir gobierno de datos, seguridad y redes	Las tres capas tienen su propia vista C4 y además están desplegadas: VPC privada, llave KMS con rotación, policy tags, escaneo de calidad en Dataplex, vistas por entidad, auditoría y Dataflow sin IP pública.	2.5
Cloud Run sobredimensionado	Lo recalculamos con la ley de Little sobre el volumen real. El servicio de autorización pasa de 2–100 a 2–20 instancias y la API de datos, de 2–100 a 0–3.	4.3
Dimensionar y validar Dataflow	Hicimos tres pruebas de carga con un worker fijo. La capacidad real resultó muy por debajo de lo que suponíamos; con ese dato fijamos el rango de workers de la fase MVP y una meta de optimización para producción.	4.2
Justificar cada componente	Para cada servicio indicamos qué hace en la solución, por qué lo elegimos y qué alternativa descartamos.	3

2 Arquitectura en notación C4

Usamos el modelo C4 porque separa los niveles de detalle: primero quién usa la plataforma y con qué sistemas se conecta (contexto) y después qué piezas la componen (contenedores). El modelo está escrito en LikeC4 (`doc/arquitectura/lumina.c4`), así que el diagrama se versiona junto con el código y se regenera con un comando. Cada componente lleva el ícono oficial del servicio de Google Cloud que representa.

2.1 Nivel 1: contexto

La plataforma recibe transacciones de los canales de las 12 entidades, cambios de saldos de sus cores bancarios, telemetría de cajeros y datos públicos. Entrega alertas al equipo antifraude, tableros a la gerencia y a cada entidad sus propios datos junto con la señal de fraude compartida.

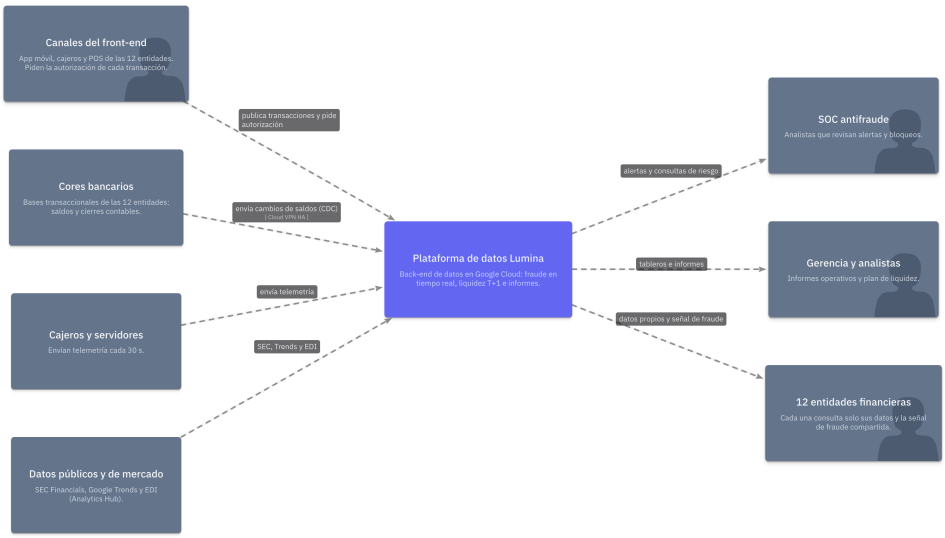


Figura 1. Diagrama de contexto (C4, nivel 1)

2.2 Nivel 2: camino streaming

Es el camino del fraude en tiempo real. La transacción entra por Pub/Sub, Dataflow la evalúa con estado por cliente y por destino, deja las *features* en Memorystore para el servicio de autorización y escribe el resultado en BigQuery. Las alertas salen por otro tópico hacia el SOC. Una copia cruda de cada evento queda en el Data Lake para auditoría y reproceso. El diagrama está en la figura 2 (página 5).

2.3 Nivel 2: camino batch, modelado y exposición

El histórico y los cierres llegan al Data Lake por Datastream y Storage Transfer. Spark en Dataproc Serverless los limpia y los deja en Parquet; de ahí pasan a Silver y, con SQL, a Gold. BigQuery ML entrena sobre Gold y devuelve el pronóstico de liquidez. Composer orquesta el proceso diario (figura 3, página 6).

2.4 Conexiones entre componentes

La numeración coincide con la de los diagramas.

Tabla 2. Conexiones de la arquitectura

N.°	Origen → destino	Qué viaja	Canal
1	Canales → Pub/Sub `tx-eventos`	Evento de transacción en JSON	gRPC con TLS, esquema Avro en el tópico
2	Pub/Sub → Dataflow	El mismo evento	Suscripción *pull*, deduplicación por `event_id`
3	Dataflow → Memorystore	Último score, ubicación y conteo de 24 h por cliente	Redis dentro de la VPC
4	Dataflow → BigQuery Silver	Transacción con score, decisión y latencia	Storage Write API
5	Dataflow → Pub/Sub `alertas-fraude`	Transacciones REVISAR o BLOQUEAR	Publicación en Pub/Sub

N.°	Origen → destino	Qué viaja	Canal
6	Pub/Sub → Data Lake	Evento crudo	Suscripción a Cloud Storage cada 5 min
7	Terminales → Pub/Sub → Bronze	Telemetría	Suscripción BigQuery, sin código
8	Cores → Datastream → Data Lake	Cambios de saldos y cierres (CDC)	Cloud VPN de alta disponibilidad
9	Data Lake ↔ Dataproc	CSV crudo de entrada, Parquet curado de salida	Conector de GCS, workers sin IP pública
10	Data Lake → Silver	`transacciones_batch`	`bq load` de Parquet
11	Datos públicos → BigQuery SQL	SEC, Google Trends y EDI	Consulta directa y Analytics Hub
12	Silver y Bronze → SQL → Gold	Dimensiones y hechos	SQL dentro de BigQuery
13	Gold → BigQuery ML	Series de efectivo y transacciones etiquetadas	`CREATE MODEL`, `ML.FORECAST`
14	Memorystore → autorización	*Features* del cliente	Direct VPC egress
15	Silver y Gold → API de datos	Alertas, riesgo, liquidez y KPI	Consultas parametrizadas
16	Gold → Looker Studio	Vistas `v_kpi_*`	Conector de BigQuery con BI Engine
17	Gold → 12 entidades	Datos propios y señal de fraude	Vistas autorizadas y filtro por entidad
18	Pub/Sub → SOC	Alertas	Suscripción *push*

2.5 Capas de gobierno, seguridad y redes

Cada capa tiene su vista. Arriba están los controles y abajo, los componentes sobre los que actúan.

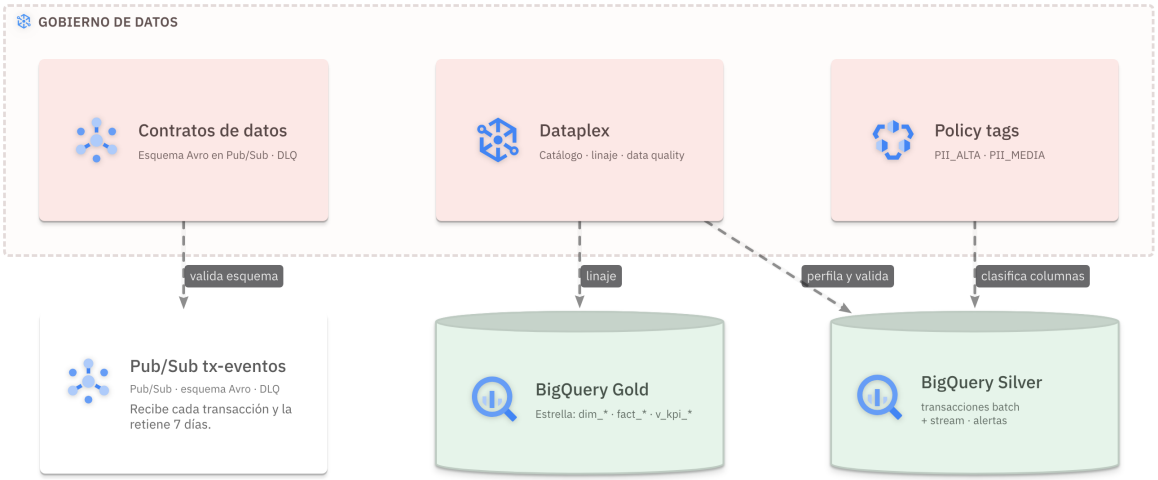


Figura 4. Capa de gobierno de datos

- **Dataplex** cataloga las tablas, registra el linaje y corre el escaneo de calidad `lumina-dq-transacciones`: unicidad de `event_id`, nulos en `cliente_id`, montos no negativos, tipos válidos y distritos entre 1 y 8.

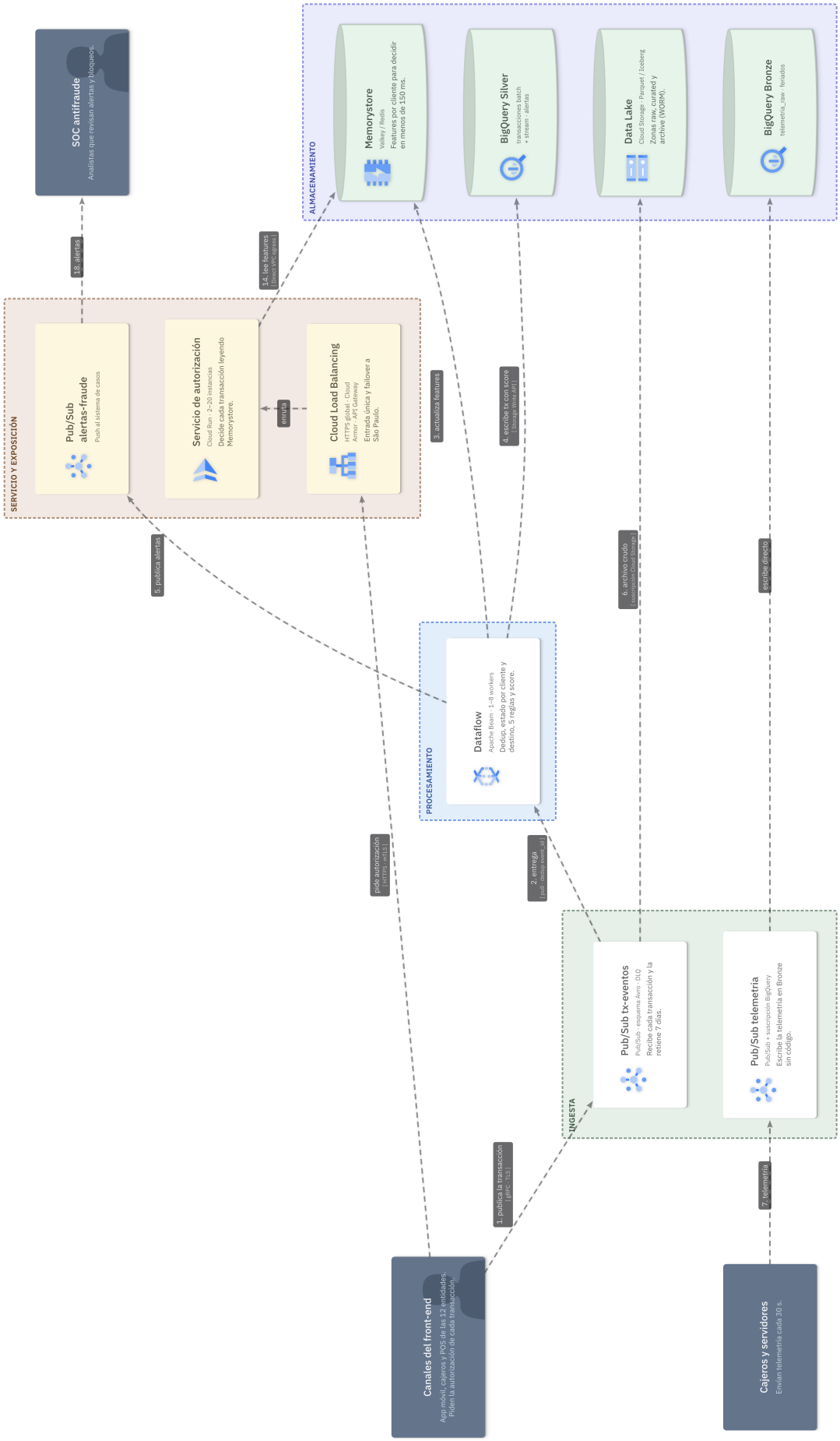


Figura 2. Contenedores del camino streaming (C4, nivel 2)

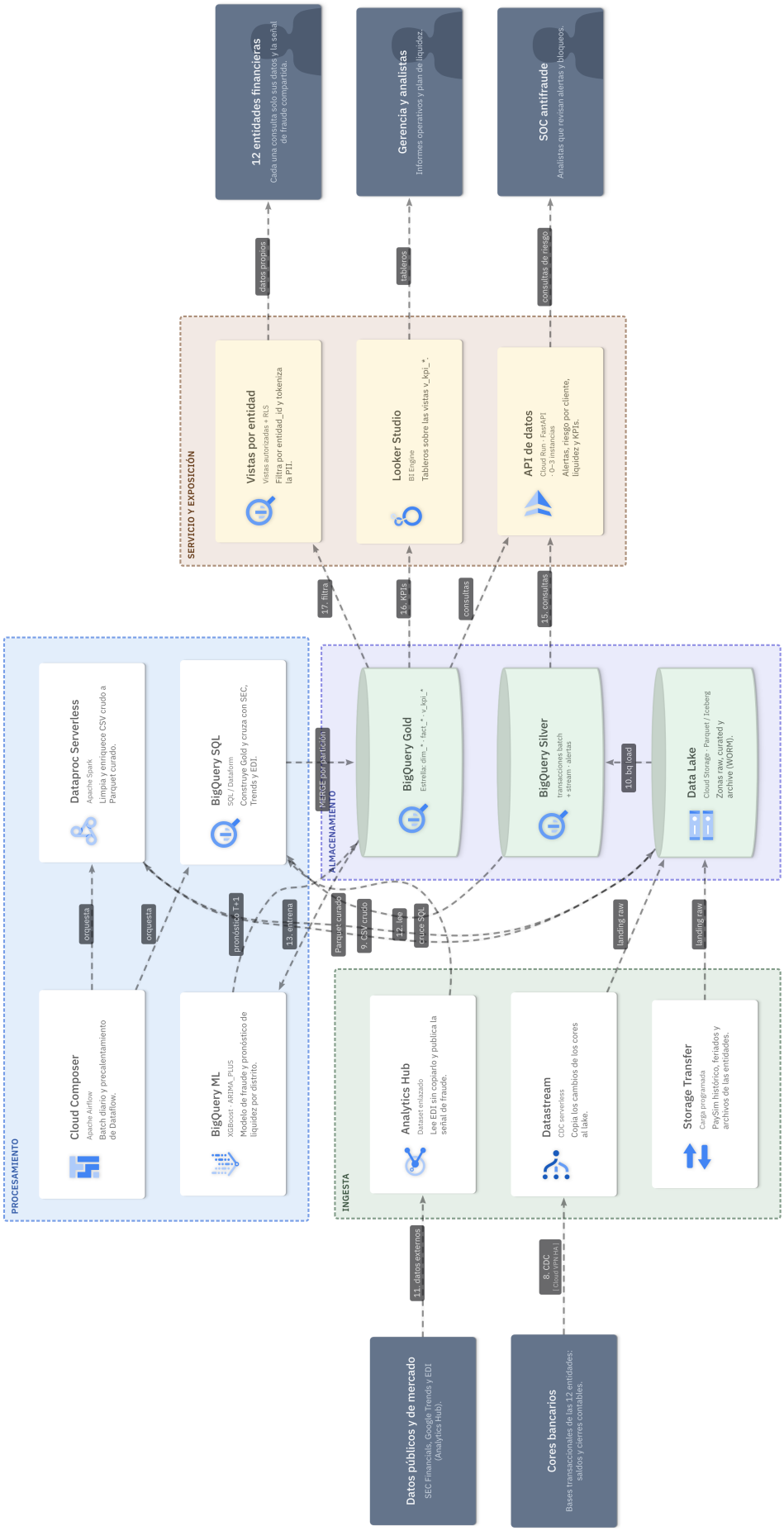


Figura 3. Contenedores del camino batch y de exposición (C4, nivel 2)

- **Policy tags**: la taxonomía `lumina_sensibilidad` distingue `PII_ALTA` (identificadores y saldos) de `PII_MEDIA` (ubicación y dispositivo).
- **Contratos de datos**: el esquema del tópico rechaza mensajes mal formados y lo que Dataflow no puede leer va a una cola de errores.
- **Señal de fraude compartida**: la vista `v_senal_fraude_compartida` publica destinos sospechosos sin datos personales. Responde a la queja del caso de que las entidades no comparten información de fraude.



Figura 5. Capa de seguridad

- **IAM**: una cuenta de servicio por componente, con roles acotados.
- **Cloud KMS**: la llave `lumina-datos` rota cada 90 días y cifra el dataset Gold y el bucket de archivo.
- **Datos personales**: la vista `v_transacciones_entidad` filtra por la entidad del usuario, reemplaza el id del cliente por un seudónimo y redondea la ubicación.
- **Auditoría**: un sink manda los Cloud Audit Logs al dataset `lumina_auditoria`.

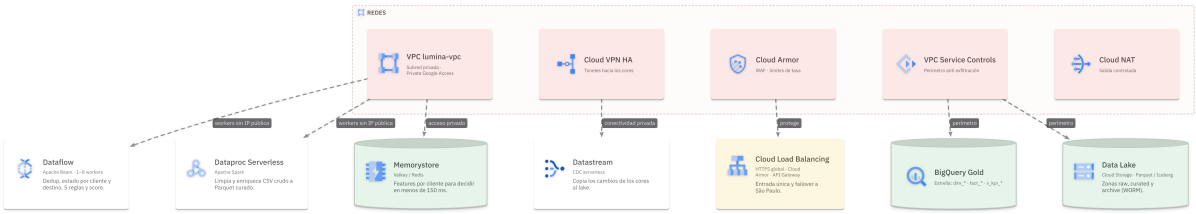


Figura 6. Capa de redes

- **VPC `lumina-vpc`** con subred privada y Private Google Access. Los workers de Dataflow corren sin IP pública dentro de ella; lo probamos en la segunda corrida del pipeline.
- **Cloud VPN** lleva el tráfico de Datastream hacia los cores sin pasar por Internet.
- **VPC Service Controls** pone un perímetro alrededor de BigQuery, Cloud Storage, Pub/Sub y Dataflow.
- **Cloud Load Balancing y Cloud Armor** son la única entrada pública y hacen el *failover* hacia São Paulo.

3 Por qué elegimos cada componente

Los criterios fueron cinco: la latencia que pide cada caso de uso, el volumen y su variación, el requisito de código abierto del caso, el trabajo de operación que le exige al equipo y el costo.

Tabla 3. Justificación técnica por componente

Componente	Qué hace en Lumina	Por qué este	Descartamos
Ingesta			
Pub/Sub	Recibe transacciones y telemetría y las retiene 7 días	Escala de 175 a 5 000 TPS sin dimensionar particiones. Tiene suscripciones nativas a BigQuery y Cloud Storage que nos ahorran dos pipelines	Kafka gestionado: hay que dimensionar brokers y particiones
Datastream	Copia los cambios de los cores	CDC sin instalar agentes en las bases de las entidades	Debezium: hay que operar Kafka Connect
Storage Transfer	Carga PaySim, feriados y archivos de las entidades	Programado, con reintentos y verificación	Scripts en una VM
Analytics Hub	Lee EDI y publica la señal de fraude	Consume datos del proveedor sin copiarlos y comparte datos entre entidades con permisos	Copiar archivos
Procesamiento			
Dataflow (Apache Beam)	Evalúa el fraude en tiempo real	Estado y *timers* gestionados para viaje imposible y cuenta mula; escala horizontal. Beam también corre en Flink o Spark	Spark Streaming: micro-lotes y clúster permanente
Dataproc Serverless (Spark)	ETL batch de CSV a Parquet	Spark es el estándar y lo conocemos; se paga por job	Clúster fijo: se paga aunque no corra
BigQuery SQL	Construye Gold y cruza con datos públicos	Transforma donde está el dato, sin moverlo	Hacer todo en Spark
BigQuery ML	Fraude (XGBoost) y liquidez (ARIMA_PLUS)	Entrena con SQL sobre los mismos datos	Vertex AI desde el inicio: más trabajo para el MVP
Cloud Composer	Orquesta el batch y el precalentamiento	Airflow: dependencias, reintentos y *backfills*	Workflows: no hace *backfill*
Almacenamiento			
Memorystore	*Features* para autorizar en menos de 150 ms	Lecturas de submilisegundos; Valkey y Redis son abiertos	Bigtable: más latencia y mayor costo mínimo
Cloud Storage	Data Lake con zonas raw, curated y archive	Barato y durable, con ciclo de vida y bloqueo WORM; formatos abiertos	HDFS: une cómputo y almacenamiento
BigQuery	Bronze, Silver, Gold y modelos	Serverless, con seguridad por fila y columna y acceso directo a SEC y Trends	Hive o PostgreSQL: operación o escala
Exposición			
Cloud Run	Autorización y API de datos	Contenedores que escalan por concurrencia; la API escala a cero	GKE: un clúster que operar
Looker Studio	Tableros de gerencia	Sin licencia; con BI Engine responde en menos de un segundo	Looker: licencia
Vistas autorizadas	Datos por entidad	Control nativo sin copiar datos por banco	Un dataset por entidad

Componente	Qué hace en Lumina	Por qué este	Descartamos
Capas transversales			
Dataplex	Catálogo, linaje y calidad	Integrado con BigQuery y Dataflow sin instalar nada	DataHub o Atlas: hay que desplegarlos
Cloud KMS	Llaves propias con rotación	Permite revocar el acceso; lo pide la SBS para datos en la nube	Llaves gestionadas por Google
VPC Service Controls	Perímetro de datos	Impide copiar datos fuera aunque la credencial sea válida	Solo IAM
Load Balancing + Armor	Entrada pública y *failover*	Balanceo global entre Santiago y São Paulo, con WAF	Balanceo regional

4 Dimensionamiento de Dataflow y Cloud Run

4.1 Volumen de referencia

Tabla 4. Escenarios de carga

Escenario	Cálculo	TPS
Promedio diario	15 M de transacciones ÷ 86 400 s	175
Hora pico de un día normal	2,5 × el promedio	440
Día de pago o cierre de mes	60 M × 12 % en la hora pico ÷ 3 600 s	2 000
Pico de diseño (Cyber Days)	2,5 × el día de pago	5 000

En la Entrega 1 sumamos la telemetría y el *clickstream* a la carga de Dataflow. Fue un error: la telemetría entra a BigQuery por una suscripción directa y el *clickstream* no pasa por el pipeline de fraude. Por Dataflow solo pasan transacciones.

4.2 Dataflow: lo que medimos

Al empezar esta entrega estimamos, sin medir, que un worker `n2-standard-2` procesaría unas 1 000 transacciones por segundo. Con eso bastaban 8 workers para el pico de diseño. Antes de fijar ese número lo pusimos a prueba.

Cómo medimos. Dejamos el job fijo en un solo worker (`--max_num_workers 1`) y publicamos transacciones de PaySim por escalones de tres minutos: 100, 300, 600 y 900 por segundo. En cada minuto registramos cuántos eventos llegaron a BigQuery, la latencia p95 entre el envío y la decisión, el backlog de la suscripción y el uso de CPU del worker. La consulta está en `src/sql/06_validacion_capacidad.sql`.

Qué encontramos. El worker se saturó en el primer escalón. Hicimos tres corridas, cambiando una cosa a la vez (tabla 5 y figura 7).

Tabla 5. Corridas de la prueba de carga con un worker `n2-standard-2`

Corrida	Cambio	ev/s sostenidos	CPU	Qué aprendimos
1	Configuración por defecto	65–75 (pico de 143)	93–97 %	El worker no sigue el ritmo: el backlog llegó a 162 000 mensajes y la latencia p95, a casi 9 minutos. Los logs mostraban lecturas de estado lentas.
2	64 hilos por proceso, dentro de la VPC privada	55–63	94–96 %	Más hilos no ayudan. El límite no es la espera por el estado sino la CPU.
3	Escritura a BigQuery en lotes (`with_auto_sharding`)	78–89	—	Mejora cerca de un 15 %. El sink no era el problema principal.

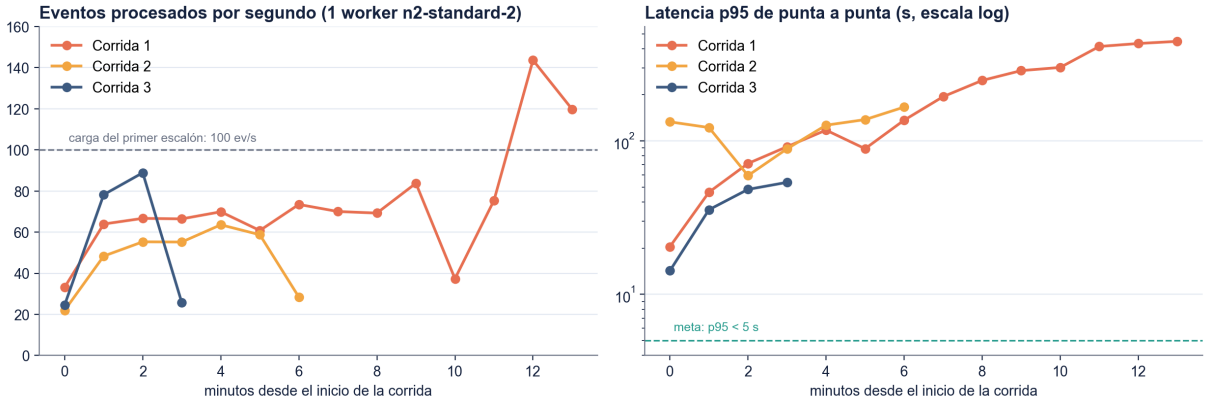


Figura 7. Eventos procesados y latencia p95 por minuto en las tres corridas

Con poca carga el pipeline sí cumple: a 5 eventos por segundo la latencia p95 fue de 2,4 s, por debajo de la meta de 5 s. El problema aparece al saturarse. Haciendo la cuenta (2 vCPU al 95 % para unos 70 eventos por segundo), cada transacción consume unos 27 ms de CPU. El costo está en las dos etapas con estado: por cada evento hay una lectura, una escritura y un *timer* por cliente y lo mismo por destino, y en el SDK de Python cada una de esas operaciones se serializa y viaja entre el proceso de Python y el *runner*. Escribimos ese código pensando en que fuera claro, no en que fuera rápido, y la prueba lo dejó a la vista.

Qué significa para el dimensionamiento. Con la capacidad medida (75 eventos por segundo, usada al 70 %), el código actual necesita 4 workers en promedio, 39 en un día de pago y 96 en el pico de diseño. Con la meta de optimización que fijamos más abajo, harían falta 1, 6 y 15 (figura 8).

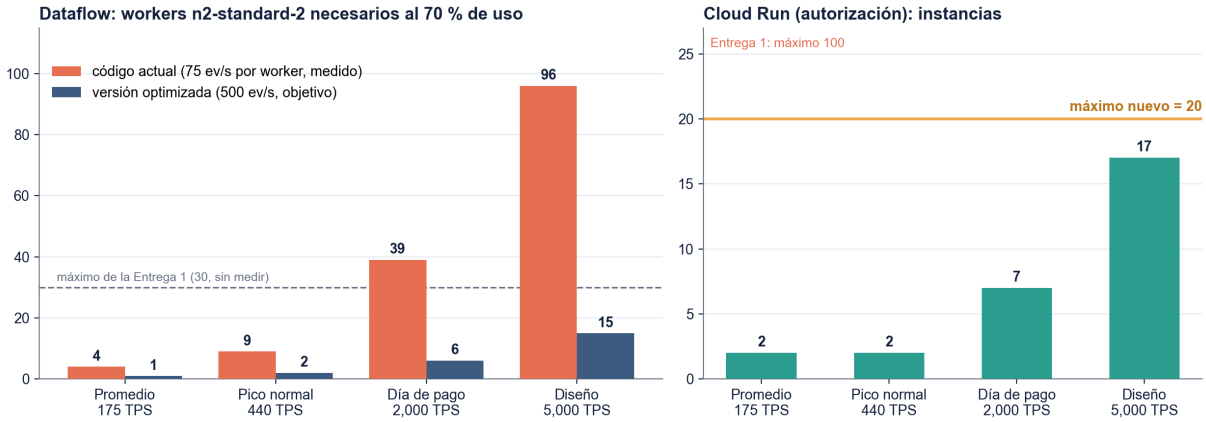


Figura 8. Workers de Dataflow e instancias de Cloud Run por escenario

Esto cambia la lectura de la Entrega 1. Allí el máximo de 30 workers `n2-standard-4` no salió de una medición: sobrestimaba la carga (contaba la telemetría) y suponía una capacidad por worker que el código no alcanza. Ahora los dos datos están medidos o corregidos, y la decisión es esta:

1. **Fase MVP, con el código actual**: mínimo de 4 workers, máximo de 40 y precalentamiento a 20 en días de pago con el DAG `dag_lumina_precalentamiento_dataflow`. Cubre hasta el día de pago. Si llegara un pico mayor, Pub/Sub retiene los mensajes sin perderlos, pero las alertas saldrían con minutos de retraso. Por eso esta versión no debe operar en un Cyber Day.
2. **Antes de producción**: optimizar el camino con estado hasta al menos 500 eventos por segundo por worker y repetir esta misma prueba. Las opciones, en el orden en que las probaremos: reescribir las dos etapas con estado en Beam para Java (el grafo y el diseño no cambian, y Beam corre igual en Dataflow); quitar el *timer* que se reprograma en cada evento y usar expiración de estado; y leer el historial del destino desde Memorystore en lugar de mantenerlo en el pipeline. Con 500 eventos por segundo, el máximo queda en 15 workers.

La autorización de cada transacción no depende de esto: la decide Cloud Run con las *features* de Memorystore en milisegundos. Dataflow mantiene esas *features* al día y genera las alertas, así que un atraso suyo degrada la calidad de la detección pero no detiene los pagos.

4.3 Cloud Run

Usamos la ley de Little: solicitudes simultáneas = tasa de llegada × tiempo de respuesta. Las instancias quedan limitadas por la CPU o por la concurrencia, lo que llegue primero, y el autoscaler de Cloud Run apunta a un 60 % de uso.

Servicio de autorización. Hay una solicitud por transacción. Cada una tarda unos 25 ms (una lectura en Memorystore dentro de la VPC más las reglas) y consume unos 2 ms de CPU, así que una instancia de 1 vCPU atiende unas 500 solicitudes por segundo; al 60 % de uso quedan 300. En el pico de diseño hacen falta $\lceil 5\,000/300 \rceil = 17$ instancias y en un día de pago, 7. Fijamos un máximo de 20 (17 más un margen del 15 %) y un mínimo de 2, para no tener arranques en frío y seguir funcionando si cae una zona. La concurrencia baja de 80 a 40: con 25 ms por solicitud cada instancia tiene unas 8 en curso, y 80 no aportaba nada.

API de datos. La usan el SOC y las aplicaciones para consultas de riesgo; los tableros van directo a BigQuery. Estimamos 20 solicitudes por segundo en el pico y 1,5 s por consulta, o sea 30 solicitudes simultáneas. Como cada solicitud pasa casi todo el tiempo esperando a BigQuery, una instancia aguanta 20 en paralelo. Con 2 instancias basta; dejamos un máximo de 3, que además limita a 60 las consultas simultáneas contra BigQuery. El mínimo es 0 en el laboratorio y 1 en producción.

Tabla 6. Configuración de Cloud Run antes y después

Parámetro	Entrega 1	Autorización	API de datos
Instancias mínimas	2	2	0 (laboratorio), 1 (producción)
Instancias máximas	100	20	3
Concurrencia	80	40	20
CPU y memoria	—	1 vCPU, 512 MiB, CPU siempre asignada	1 vCPU, 512 MiB, CPU solo durante la solicitud

Bajar el máximo no reduce el costo por sí solo, porque Cloud Run cobra por uso. Lo que hace es poner un techo al gasto y proteger a Memorystore y BigQuery de una avalancha de solicitudes. La API ya está desplegada con esta configuración (figura 53).

4.4 Impacto en el costo

Recalculamos el costo mensual con precios de lista de `us-central1`, igual que en la Entrega 1, y lo llevamos a Santiago con un ajuste del 22 %. La tabla 7 muestra solo las partidas que cambian; el resto (Pub/Sub, Dataproc, BigQuery, Memorystore, Cloud Storage, Composer, seguridad, observabilidad y DR) suma USD 2 635 al mes y no se modifica.

Tabla 7. Costo mensual estimado (USD)

Partida	Entrega 1	MVP (código actual)	Producción (optimizado)
Dataflow	1 100	800	250
Cloud Run, balanceador y Cloud Armor	80	160	160
Redes (VPN hacia los cores, NAT, egress)	80	180	180
Resto de servicios	2 635	2 635	2 635
Subtotal más 15 % de contingencia	4 480	4 340	3 710
Santiago con DR en São Paulo	**5 500**	**5 300**	**4 500**
Año 1	**66 000**	**63 600**	**54 300**
USD por millón de transacciones	12	12	10

Dataflow en el MVP supone unos 4,5 workers en promedio a lo largo del día, más los picos de los días de pago. Cloud Run sube respecto de la Entrega 1 porque ahora contamos bien las dos instancias mínimas del servicio de autorización, y la partida de redes crece porque incluye la VPN hacia los cores. Son estimaciones que hay que confirmar en la calculadora de precios de Google Cloud.

El laboratorio corre bajo un presupuesto de S/ 180 con alertas al 50, 90 y 100 % (figura 9). Dataflow se apagó al terminar cada prueba; la API escala a cero cuando nadie la usa.

5 Implementación de la Entrega 2

5.1 Pipeline batch con Spark

El script `src/dataproc/batch_paysim_etl.py` lee el CSV de PaySim con un esquema explícito y escribe Parquet particionado por fecha. Descarta filas sin llaves o monto, tipos fuera de dominio,

montos negativos y duplicados. Genera un `event_id` determinista, convierte el paso de simulación en fecha real y asigna distrito, entidad, edad, canal y ubicación con el mismo hash MD5 que usan el simulador y el SQL. También calcula los descuadres de saldo, que son la señal más fuerte de fraude en PaySim.

La corrida en Dataproc Serverless tardó 2 min 51 s, leyó 6 362 620 filas y las escribió todas en 31 particiones (552 MiB en Parquet). PaySim no trae errores de origen, así que no se descartó ninguna fila. Después, `bq load` llevó el Parquet a `lumina_silver.transacciones_batch`, particionada por día y agrupada por distrito, tipo y cliente.

Al descargar el dataset de Kaggle encontramos dos problemas que obligaron a normalizarlo antes de subirlo: viene partido en `train.csv` y `validation.csv`, con las columnas `isFraud` e `isFlaggedFraud` en orden invertido, y con fines de línea de Windows. Como el ETL lee por posición, sin esa limpieza la etiqueta de fraude habría quedado cambiada.

5.2 Pipeline streaming con Apache Beam

Tabla 8. Pasos del pipeline `fraud_streaming_pipeline.py`

Paso	Qué hace
`LeerPubSub`	Lee la suscripción y descarta duplicados por `event_id`
`Parsear`	Valida campos, tipos y montos; lo inválido va al tópico `tx-eventos-dlq`
`ViajeImposible`	Guarda por cliente la última ubicación y calcula la velocidad. Regla R4 si supera 800 km/h
`VelocidadDestino`	Cuenta por destino las transacciones de los últimos 10 minutos. Regla R5 (cuenta mula) con 3 o más
`Reglas`	Suma R1 a R5 en un score: desde 0,8 bloquea, desde 0,5 manda a revisión
Salidas	Todas las transacciones a Silver; las alertas a Silver y al tópico `alertas-fraude`

En la primera corrida el simulador publicó 30 000 transacciones a 100 por segundo y las 30 000 llegaron a Silver sin duplicados; 130 terminaron en alerta. En paralelo, 50 terminales enviaron 6 000 lecturas de telemetría que la suscripción escribió en Bronze sin pasar por ningún pipeline.

5.3 Batch y streaming en el mismo modelo

Los dos caminos escriben en Silver con el mismo esquema y asignan distrito y entidad con la misma función. La vista `fact_transacciones` los une con una columna `origen`, de modo que un KPI calculado sobre el histórico se puede comparar con uno calculado en vivo.

5.4 Modelo estrella en Gold

Tabla 9. Tablas y vistas de la capa Gold

Objeto	Grano	Uso
`dim_distrito`	8 distritos	Liquidez y congestión; cada uno se asocia a una región de Google Trends
`dim_entidad`	12 entidades	Acceso por entidad; cada una tiene un banco de la SEC como referencia
`dim_cliente`	6,9 M de clientes	Segmentación por edad, distrito y entidad
`dim_fecha`	Día	Feriados y días de pago
`fact_transacciones`	Transacción	Une batch y streaming
`fact_liquidez_hora`	Distrito × hora	Entrada del pronóstico de efectivo
`fact_tendencias_semana`	Distrito × semana	Demanda digital según Google Trends
`fact_benchmark_sector`	Banco × año	ROA y apalancamiento del sector
`v_kpi_*` y vistas de gobierno	—	Informes operativos, reglas de fraude, acceso por entidad

Dos ajustes respecto a lo planeado. Google Trends no publica datos de Perú en `international_top_terms`, así que usamos Colombia como referencia andina (33 regiones). Y para elegir los bancos de referencia usamos los informes 10-K de 2018, el último año con cobertura completa en el dataset de la SEC; con 2020 solo aparecían 4 bancos.

5.5 Modelos en BigQuery ML

El modelo de fraude (XGBoost) se evaluó sobre el 20 % de TRANSFER y CASH_OUT que no vio en el entrenamiento, con la proporción real de fraude. Obtuvo una precisión de 0,81, un recall de 0,995 y un AUC de 0,998, por encima de la meta de la Entrega 1 (recall de al menos 85 % con precisión de al menos 70 %). La variable que más pesa es el descuadre del saldo de origen. Las reglas del pipeline, sin modelo, detectan el 98,9 % de los fraudes con una precisión de 0,52; el modelo reduce a la mitad las falsas alarmas.

El pronóstico de liquidez (ARIMA_PLUS por distrito) no alcanza la meta de un error menor al 15 %: el MAPE diario va de 32 % a 59 % en siete de los ocho distritos si se excluye el último día. La causa está en los datos: en sus últimos días PaySim casi no tiene movimiento (282 transacciones el 31 de julio frente a unas 50 000 en un día típico), y esos son justamente los días reservados para evaluar. Con un año de datos reales el modelo podrá aprender la estacionalidad semanal y mensual; con 27 días no puede.

5.6 API de datos

La API en Cloud Run responde 403 a quien llama sin credenciales y devuelve las alertas, el riesgo de un cliente y el plan de liquidez a quien presenta un token válido (figura 56). En la Entrega 3 la conectaremos al tablero y a la demo.

6 Evidencias de la implementación

Todas las capturas son salidas reales de `gcloud` y `bq` ejecutadas sobre el proyecto `lumina-bank-utec` el 9 y 10 de noviembre de 2026. Cada una muestra el comando, la fecha y el resultado. Los logs

completos están en `doc/evidencias/logs/`. Las figuras marcadas como "Consola" son capturas de la consola web de Google Cloud y complementan la salida de terminal del mismo recurso.

6.1 Proyecto e infraestructura base

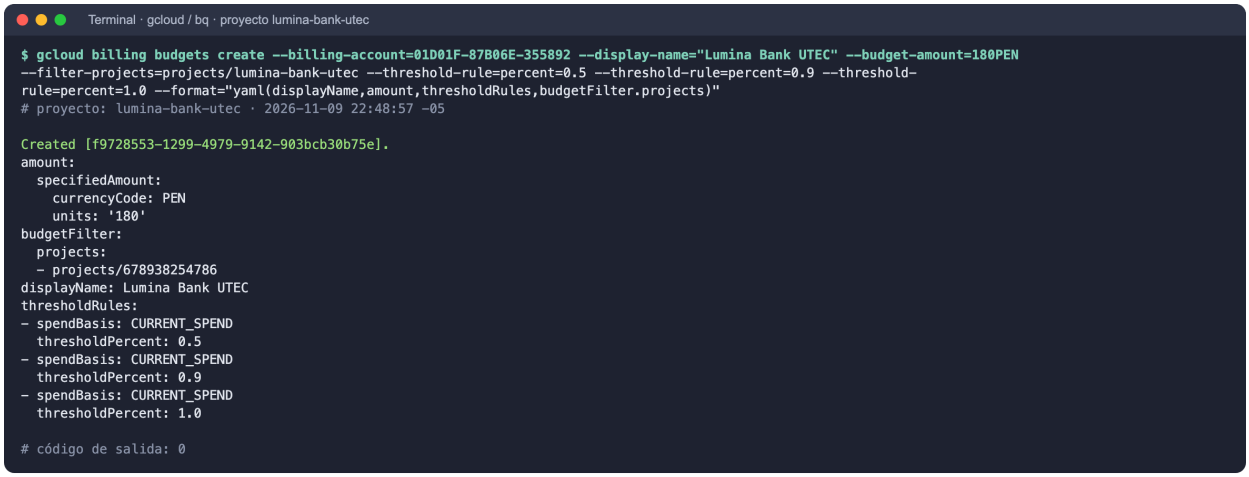


Figura 9. Presupuesto de S/ 180 con alertas al 50, 90 y 100 %. La cuenta de facturación está en soles

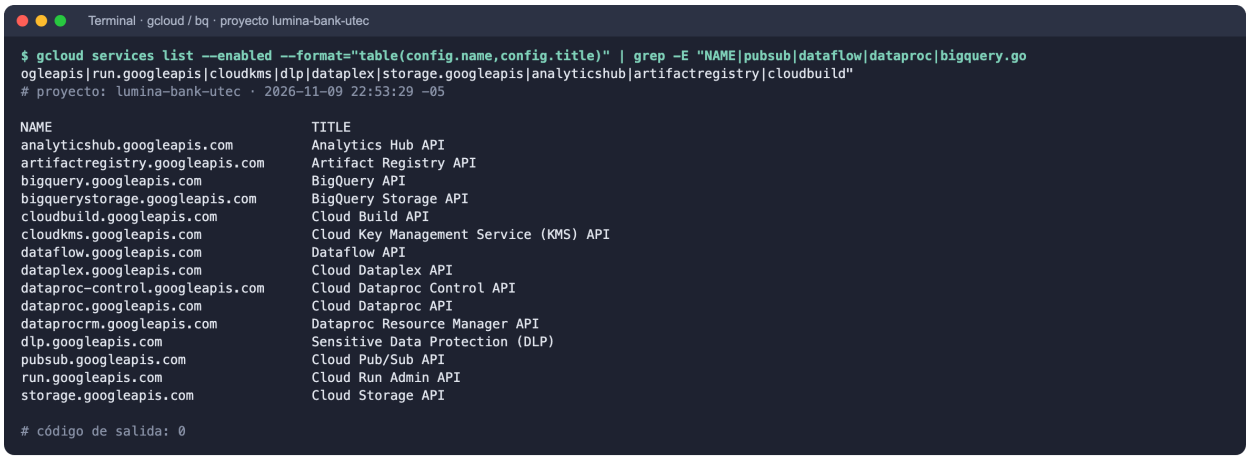


Figura 10. APIs habilitadas en el proyecto

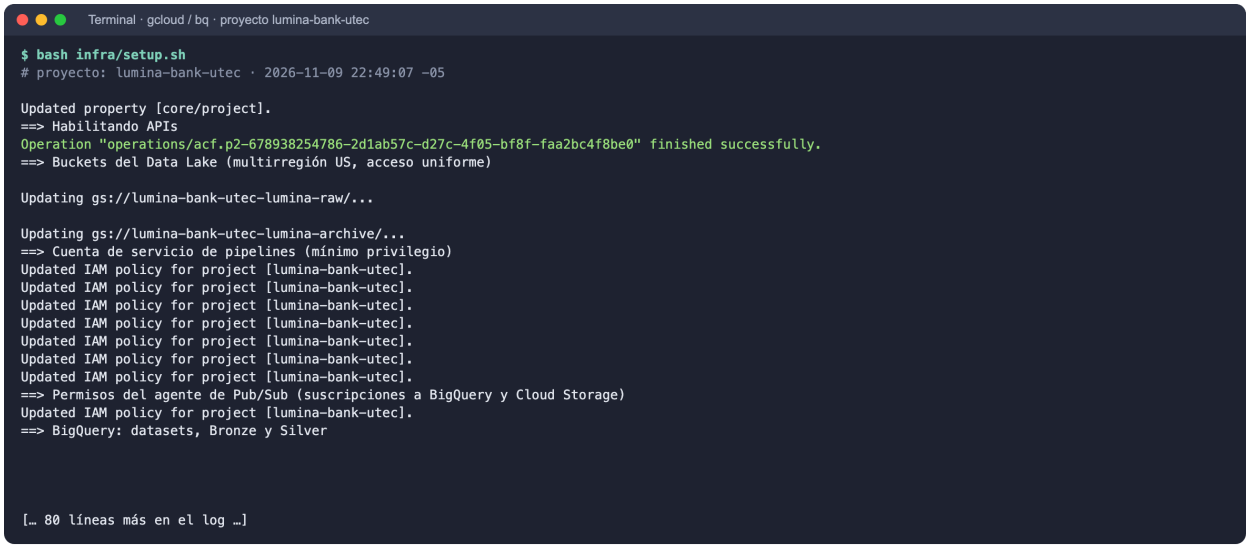


Figura 11. Ejecución de `infra/setup.sh`: APIs, buckets, cuenta de servicio y datasets

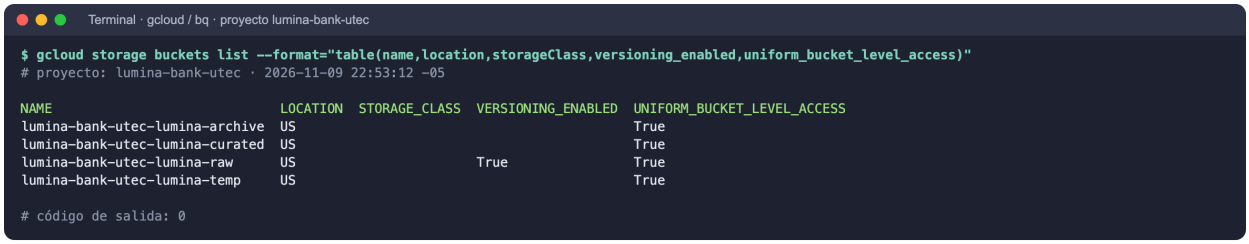


Figura 12. Los cuatro buckets del Data Lake en la multirregión US; `raw` tiene versionado

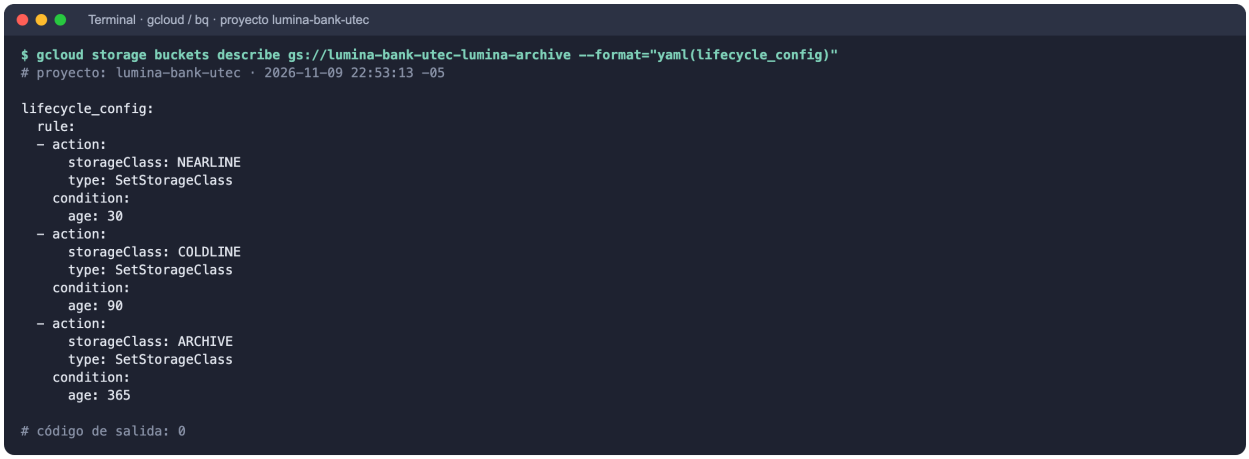


Figura 13. Ciclo de vida del bucket de archivo: Nearline a 30 días, Coldline a 90 y Archive a 365

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud pubsub topics list --format="table(name.basename(),messageRetentionDuration)"; echo; gcloud pubsub subscriptions list
--format="table(name.basename(),topic.basename(),bigqueryConfig.table,cloudStorageConfig.bucket,ackDeadlineSeconds)"
# proyecto: lumina-bank-utec · 2026-11-09 22:53:15 -05

NAME            MESSAGE_RETENTION_DURATION
tx-eventos      604800s
tx-eventos-dlq  604800s
alertas-fraude  604800s
telemetria      604800s

NAME                    TOPIC           TABLE                                       BUCKET
ACK_DEADLINE_SECONDS
tx-eventos-archivo      tx-eventos                                                  lumina-bank-utec-lumina-raw  300
tx-eventos-dlq-revision tx-eventos-dlq                                                                       10
tx-eventos-dataflow     tx-eventos                                                                           60
alertas-fraude-demo     alertas-fraude                                                                       10
telemetria-bq           telemetria      lumina-bank-utec:lumina_bronze.telemetria_raw                        10

# código de salida: 0
```

Figura 14. Tópicos y suscripciones de Pub/Sub: Dataflow, archivo en Cloud Storage y escritura directa a BigQuery

6.2 Gobierno, seguridad y redes

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bash infra/gobierno_seguridad_red.sh
# proyecto: lumina-bank-utec · 2026-11-10 00:20:03 -05

Operation "operations/acat.p2-678938254786-66d35687-239c-49e4-8123-a9229a6f6e3e" finished successfully.
==> VPC privada y subred con Private Google Access
   lumina-vpc ya existe
   lumina-datos-us-central1 ya existe
==> Cloud KMS: llave CMEK con rotación cada 90 días
   keyring ya existe
   llave ya existe

Updating gs://lumina-bank-utec-lumina-archive/...
Dataset 'lumina-bank-utec:lumina_gold' successfully updated.
==> Auditoría: logs de Cloud Audit → BigQuery (lumina_auditoria)
   lumina_auditoria ya existe
==> Policy tags (Data Catalog): taxonomía de sensibilidad
   la taxonomía ya existe (o falló la creación): revise Dataplex → Policy tags
==> Dataplex: escaneo de calidad sobre lumina_silver.transacciones_batch
   el escaneo ya existe
projects/678938254786/locations/us-central1/dataScans/lumina-dq-transacciones/jobs/e0ed8781-3b71-4096-ad50-ddd9d610be76
==> Vistas por entidad y señal de fraude compartida



CREATE TABLE IF NOT EXISTS `lumina-bank-utec.lumina_gold.acceso_entidad` (
  usuario STRING NOT NULL OPTIONS (description = 'correo del usuario (SESSION_USER)'),
  entidad_id INT64 NOT NULL
) OPTIONS (description = 'Gobierno: mapeo usuario → entidad para la vista por entidad'); -- at [7:1]
Skipped lumina-bank-utec.lumina_gold.acceso_entidad

CREATE OR REPLACE VIEW `lumina-bank-utec.lumina_gold.v_transacciones_entidad`
OPTIONS (description = 'Seguridad: filas filtradas por entidad del usuario y PII tokenizada') AS
SELECT t.event_id, t.event_ts, t.fecha, t.tipo, t.monto, t.canal, t.entidad_id, t.distrito_id,
       TO_HEX(SHA256(CONCAT('lumina-salt', t.cliente_id))) AS cliente_token,  -- seudónimo estable
       ROUND(t.lat, 2) AS lat_aprox, ROUND(t.lon, 2) AS lon_aprox,            -- geo con menor precisión
       t.score, t.decision, t.origen
FROM `lumina-bank-utec.lumina_gold.fact_transacciones` t

[… 25 líneas más en el log …]
```

Figura 15. Segunda ejecución de `infra/gobierno_seguridad_red.sh`: el script detecta lo que ya creó y no lo duplica

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud compute networks subnets describe lumina-datos-us-central1 --region us-central1
--format='yaml(name,network.basename(),ipCidrRange,privateIpGoogleAccess,logConfig.enable)'; echo; gcloud compute firewall-rules
 list --filter='network:lumina-vpc' --format='table(name,direction,sourceRanges.list(),allowed[].map().firewall_rule().list())'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:24 -05

ipCidrRange: 10.10.0.0/20
logConfig:
  enable: true
name: lumina-datos-us-central1
network: lumina-vpc
privateIpGoogleAccess: true

NAME            DIRECTION  SOURCE_RANGES    ALLOW
lumina-iap-ssh  INGRESS    35.235.240.0/20  tcp:22
lumina-interno  INGRESS    10.10.0.0/20     tcp,udp,icmp

# código de salida: 0
```

Figura 16. Subred privada con Private Google Access y reglas de firewall de `lumina-vpc`

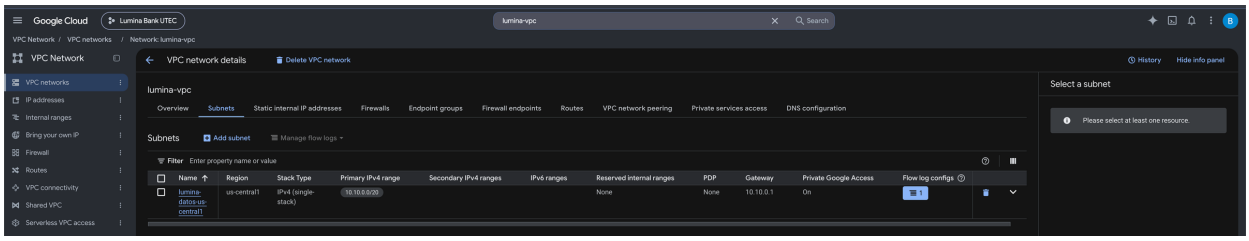


Figura 17. Consola: subred `lumina-datos-us-central1` (10.10.0.0/20) con Private Google Access activo y flow logs

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud kms keys describe lumina-datos --keyring lumina-kr --location us
--format='yaml(name,purpose,rotationPeriod,nextRotationTime,primary.state,primary.algorithm)'; echo; gcloud kms keys get-iam-
policy lumina-datos --keyring lumina-kr --location us --format='yaml(bindings)'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:27 -05

name: projects/lumina-bank-utec/locations/us/keyRings/lumina-kr/cryptoKeys/lumina-datos
nextRotationTime: '2027-02-08T04:25:48Z'
primary:
  algorithm: GOOGLE_SYMMETRIC_ENCRYPTION
  state: ENABLED
purpose: ENCRYPT_DECRYPT
rotationPeriod: 7776000s

bindings:
- members:
  - serviceAccount:bq-678938254786@bigquery-encryption.iam.gserviceaccount.com
  - serviceAccount:service-678938254786@gs-project-accounts.iam.gserviceaccount.com
  role: roles/cloudkms.cryptoKeyEncrypterDecrypter

# código de salida: 0
```

Figura 18. Llave `lumina-datos`: rotación cada 90 días y permisos solo para los agentes de BigQuery y Cloud Storage

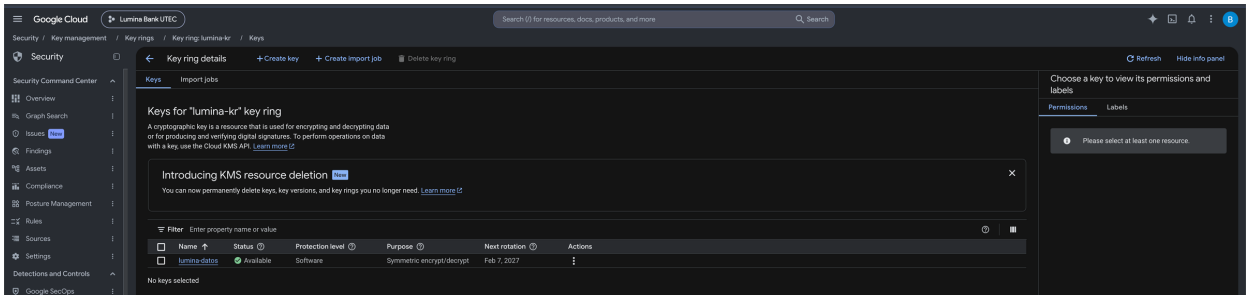


Figura 19. Consola: llave `lumina-datos` disponible en el keyring `lumina-kr`, con la próxima rotación programada para el 7 de febrero de 2027

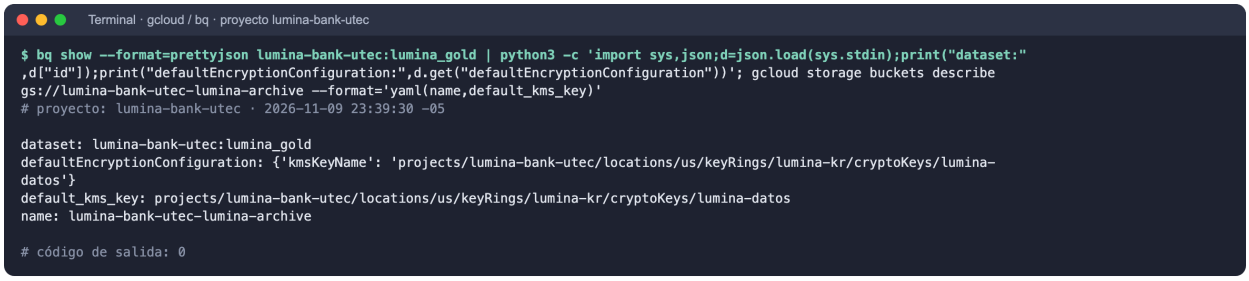


Figura 20. El dataset Gold y el bucket de archivo usan la llave propia (CMEK)

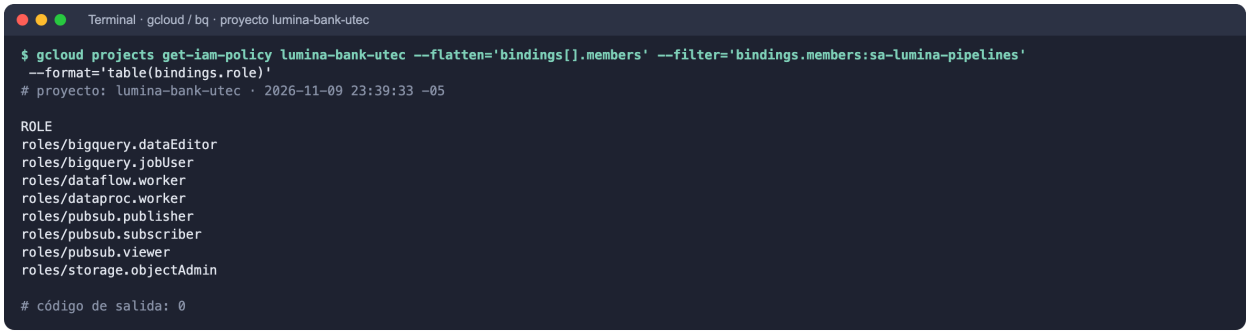


Figura 21. Roles de la cuenta de servicio de los pipelines

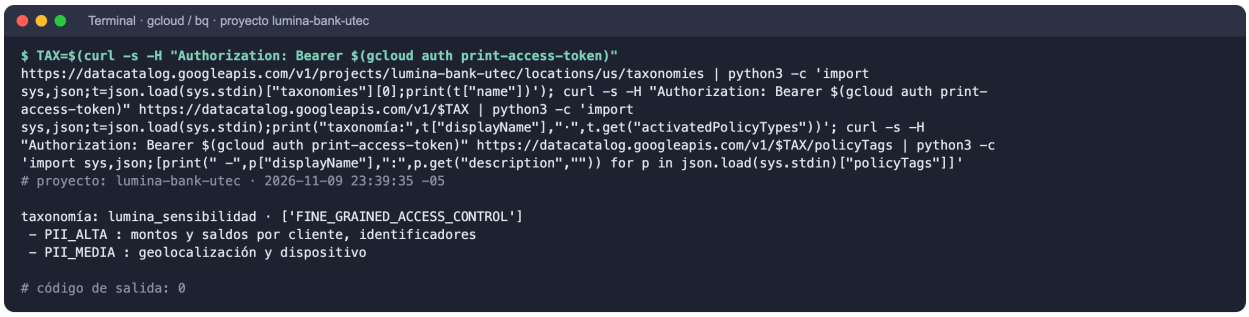


Figura 22. Taxonomía de sensibilidad con los tags `PII_ALTA` y `PII_MEDIA`



Figura 23. Escaneo de calidad de Dataplex sobre Silver: cinco reglas aprobadas en una muestra de 636 421 filas

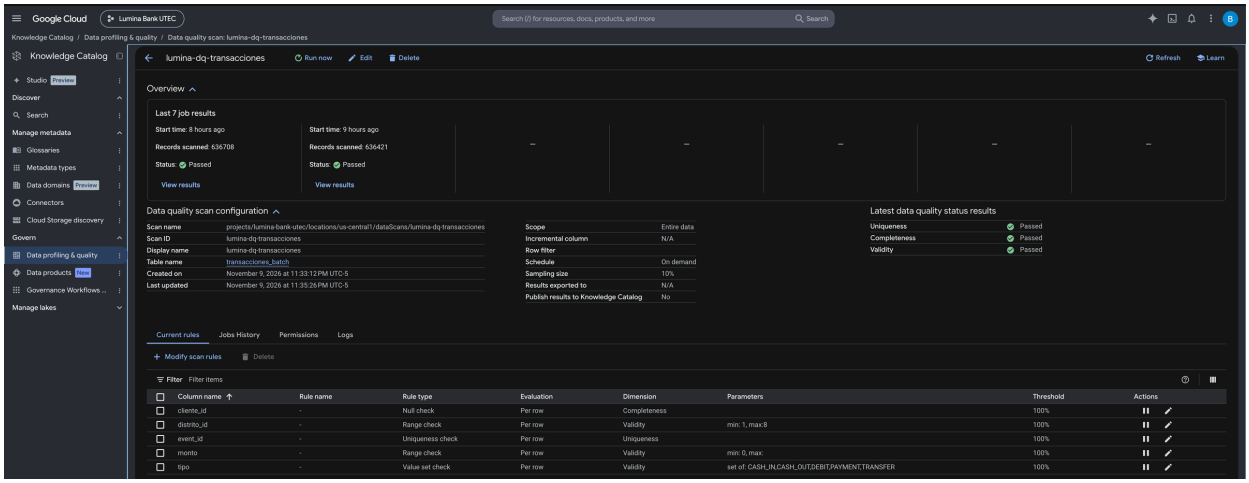


Figura 24. Consola de Knowledge Catalog: dos ejecuciones del escaneo aprobadas y las cinco reglas con umbral de 100 %

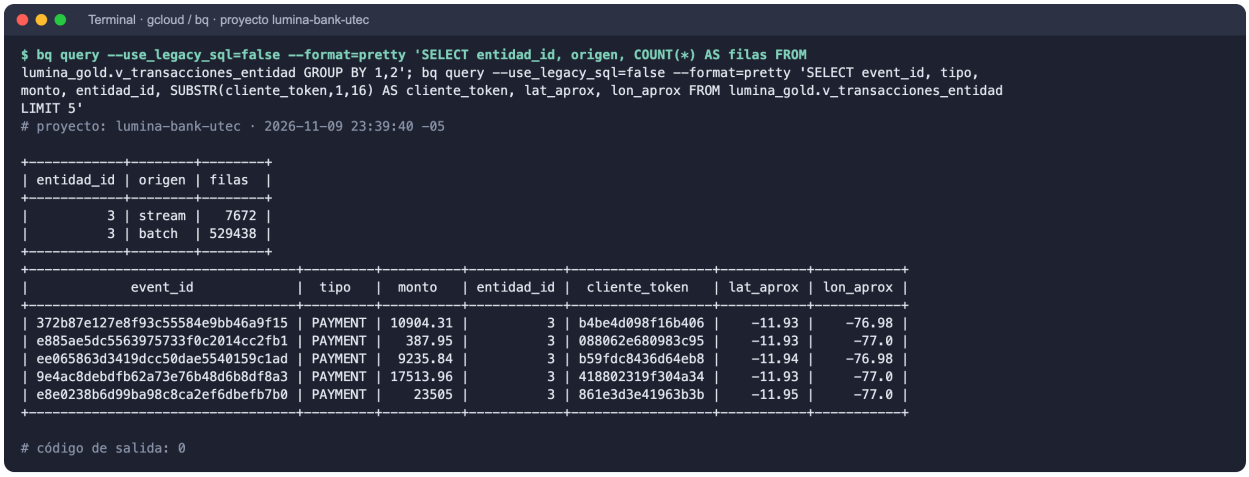


Figura 25. La vista por entidad solo devuelve filas de la entidad 3 y reemplaza el id del cliente por un seudónimo

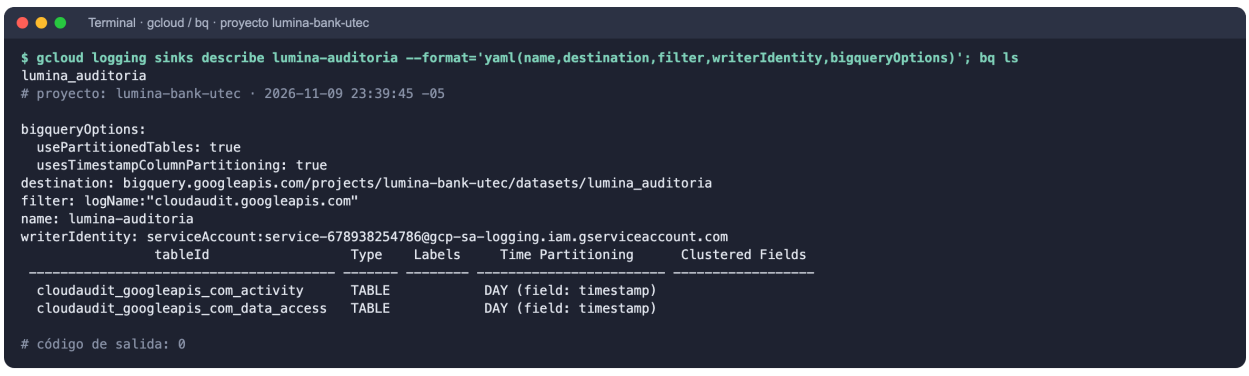


Figura 26. Sink de auditoría y tablas de Cloud Audit Logs en BigQuery

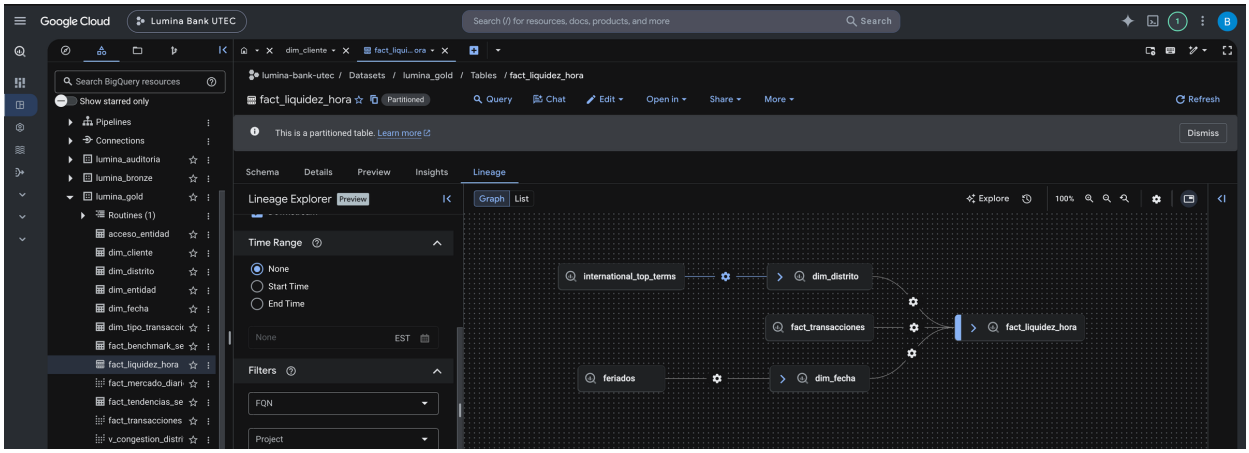


Figura 27. Linaje de `fact_liquidez_hora` en BigQuery: viene de `fact_transacciones`, `dim_distrito` (Google Trends) y `dim_fecha` (feriados). El tramo de Spark lo publica Dataproc; los pasos de BigQuery los registramos con eventos OpenLineage (`infra/registrar_linaje.py`)

6.3 Pipeline batch

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud storage ls -l gs://lumina-bank-utec-lumina-raw/paysim/
# proyecto: lumina-bank-utec · 2026-11-09 23:02:45 -05

 492534846  2026-11-10T04:02:36Z  gs://lumina-bank-utec-lumina-raw/paysim/paysim.csv
TOTAL: 1 objects, 492534846 bytes (469.72MiB)

# código de salida: 0
```

Figura 28. PaySim normalizado en la zona raw del Data Lake (476 MiB)

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud dataproc batches submit pyspark gs://lumina-bank-utec-lumina-temp/code/batch_paysim_etl.py --batch lumina-
etl-20261110-v2 --region us-central1 --version 2.2 --service-account sa-lumina-pipelines@lumina-bank-
utec.iam.gserviceaccount.com --deps-bucket gs://lumina-bank-utec-lumina-temp -- --entrada gs://lumina-bank-utec-lumina-
raw/paysim/ --salida gs://lumina-bank-utec-lumina-curated/transacciones/
# proyecto: lumina-bank-utec · 2026-11-09 23:02:48 -05

Batch [lumina-etl-20261110-v2] submitted.
Using the default container image
time="2026-11-10T04:03:47Z" level=warning msg="cgroup_parent is not set. Use `runtime-config` to get the runtime cgroup driver"
time="2026-11-10T04:03:47Z" level=warning msg="cgroup_parent is not set. Use `runtime-config` to get the runtime cgroup driver"
Waiting for container log creation
PYSPARK_PYTHON=/opt/dataproc/conda/bin/python
Generating /home/spark/.pip/pip.conf
Configuring index-url as 'https://us-python.pkg.dev/artifact-registry-python-cache/virtual-python/simple/'
JAVA_HOME=/usr/lib/jvm/temurin-17-jdk-amd64
SPARK_EXTRA_CLASSPATH=
:: loading settings :: file = /etc/spark/conf/ivysettings.xml
26/11/10 04:03:55 WARN MetricsConfig: Cannot locate configuration: tried hadoop-metrics2-google-hadoop-file-
system.properties,hadoop-metrics2.properties
26/11/10 04:03:55 INFO MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).

[… 18 líneas más en el log …]
```

Figura 29. Envío del job de Spark a Dataproc Serverless con la línea de control de calidad

```
Terminal · gcloud / bq · proyecto lumina-bank-utec
$ gcloud dataproc batches describe lumina-etl-20261110-v2 --region us-central1 --format="yaml(name.basename(),state,stateTime,cr
eateTime,runtimeConfig.version,environmentConfig.executionConfig.serviceAccount,runtimeInfo.approximateUsage)"
# proyecto: lumina-bank-utec · 2026-11-09 23:06:14 -05

createTime: '2026-11-10T04:02:49.578829Z'
environmentConfig:
  executionConfig:
    serviceAccount: sa-lumina-pipelines@lumina-bank-utec.iam.gserviceaccount.com
name: lumina-etl-20261110-v2
runtimeConfig:
  version: 2.2.90
state: SUCCEEDED
stateTime: '2026-11-10T04:05:40.133316Z'

# código de salida: 0
```

Figura 30. Estado final del job de Spark: SUCCEEDED en 2 min 51 s

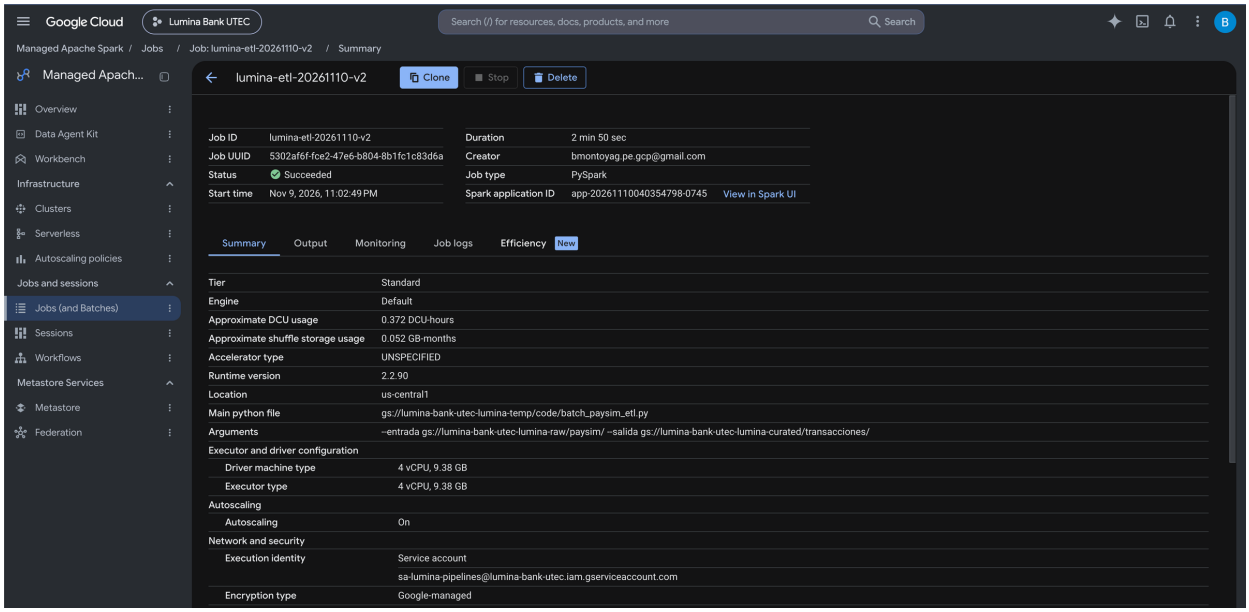


Figura 31. Consola de Dataproc Serverless: batch `lumina-etl-20261110-v2` terminado en 2 min 50 s con la cuenta de servicio de los pipelines (0,37 DCU-hora)

```
Terminal · gcloud / bq · proyecto lumina-bank-utec
$ gcloud storage ls gs://lumina-bank-utec-lumina-curated/transacciones/ | head -8; echo "..."; echo "particiones: $(gcloud
storage ls gs://lumina-bank-utec-lumina-curated/transacciones/ | grep -c fecha=)"; gcloud storage du -s -r gs://lumina-bank-
utec-lumina-curated/transacciones/
# proyecto: lumina-bank-utec · 2026-11-09 23:06:16 -05

gs://lumina-bank-utec-lumina-curated/transacciones/

gs://lumina-bank-utec-lumina-curated/transacciones/:
gs://lumina-bank-utec-lumina-curated/transacciones/
gs://lumina-bank-utec-lumina-curated/transacciones/_SUCCESS
gs://lumina-bank-utec-lumina-curated/transacciones/fecha=2026-07-01/
gs://lumina-bank-utec-lumina-curated/transacciones/fecha=2026-07-02/
gs://lumina-bank-utec-lumina-curated/transacciones/fecha=2026-07-03/
particiones: 31
551.98MiB    gs://lumina-bank-utec-lumina-curated/transacciones/

# código de salida: 0
```

Figura 32. Zona curated: 31 particiones por fecha en Parquet

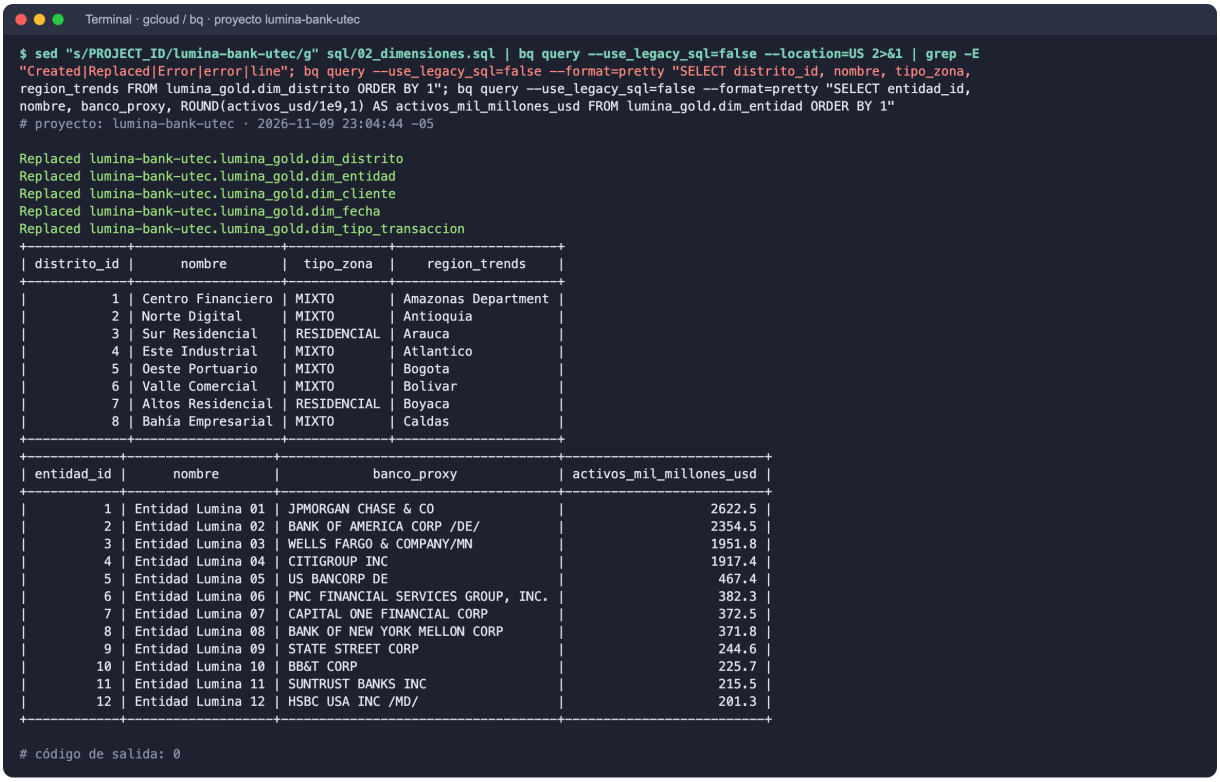


Figura 33. Dimensiones conformadas: distritos asociados a regiones de Trends y entidades con su banco de referencia en la SEC

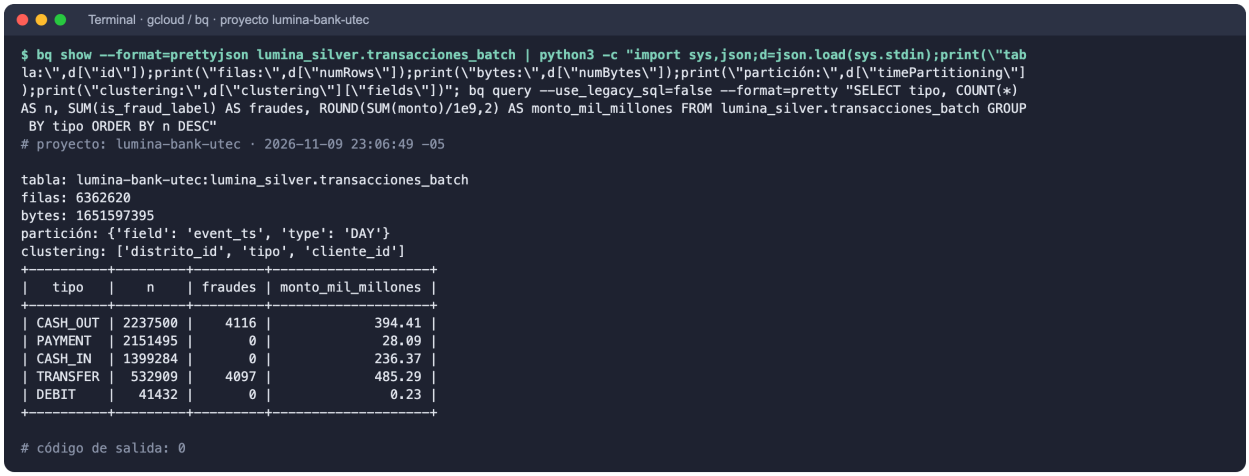


Figura 34. Tabla Silver de transacciones batch: 6 362 620 filas, particionada y agrupada

6.4 Pipeline streaming

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ python publish_transactions.py --project lumina-bank-utec --csv paysim.csv --solo-transfer-cashout --rate 100 --limit 30000
# proyecto: lumina-bank-utec · 2026-11-09 23:07:27 -05

1000 eventos publicados (100 ev/s)
2000 eventos publicados (100 ev/s)
3000 eventos publicados (100 ev/s)
4000 eventos publicados (100 ev/s)
5000 eventos publicados (100 ev/s)
6000 eventos publicados (100 ev/s)
7000 eventos publicados (100 ev/s)
8000 eventos publicados (100 ev/s)
9000 eventos publicados (100 ev/s)
10000 eventos publicados (100 ev/s)
11000 eventos publicados (100 ev/s)

[… 19 líneas más en el log …]
```

Figura 35. Simulador publicando 30 000 transacciones a 100 por segundo

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bq query --use_legacy_sql=false --format=pretty 'SELECT decision, COUNT(*) AS n, ROUND(AVG(score),2) AS score_prom FROM
lumina_silver.transacciones_stream WHERE event_ts BETWEEN "2026-11-10 04:07:00" AND "2026-11-10 04:13:00" GROUP BY 1 ORDER BY 2
DESC'; bq query --use_legacy_sql=false --format=pretty 'SELECT FORMAT_TIMESTAMP("%H:%M:%S", event_ts) AS hora, tipo, monto,
distrito_id AS dist, score, decision, ARRAY_TO_STRING(reglas, ",") AS reglas FROM lumina_silver.alertas_fraude WHERE event_ts
BETWEEN "2026-11-10 04:07:00" AND "2026-11-10 04:13:00" ORDER BY score DESC, event_ts LIMIT 8'; bq query --use_legacy_sql=false
--format=pretty 'SELECT COUNT(*) AS lecturas, COUNT(DISTINCT terminal_id) AS terminales, MIN(publish_time) AS desde,
MAX(publish_time) AS hasta FROM lumina_bronze.telemetria_raw WHERE DATE(publish_time) = "2026-11-10"'
# proyecto: lumina-bank-utec · 2026-11-10 00:09:07 -05

+----------+-------+------------+
| decision |   n   | score_prom |
+----------+-------+------------+
| APROBAR  | 29862 |       0.03 |
| REVISAR  |    72 |        0.5 |
| BLOQUEAR |    50 |       0.97 |
+----------+-------+------------+
+----------+----------+------------+------+-------+----------+-----------------------------------------------------------------------------------------+
|   hora   |   tipo   |   monto    | dist | score | decision |                                          reglas                                         |
+----------+----------+------------+------+-------+----------+-----------------------------------------------------------------------------------------+
| 04:07:31 | TRANSFER | 1735466.52 |    5 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R2_INCONSISTENCIA_DESTINO,R3_MONTO_ALTO                               |
| 04:07:31 | TRANSFER |  316877.85 |    1 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R2_INCONSISTENCIA_DESTINO,R3_MONTO_ALTO                               |
| 04:07:32 | CASH_OUT |    78810.3 |    8 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R4_VIAJE_IMPOSIBLE                                                    |
| 04:07:35 | TRANSFER | 2542664.27 |    6 |   1.0 | BLOQUEAR | R2_INCONSISTENCIA_DESTINO,R3_MONTO_ALTO,R4_VIAJE_IMPOSIBLE                              |
| 04:07:36 | TRANSFER | 1273674.05 |    5 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R2_INCONSISTENCIA_DESTINO,R3_MONTO_ALTO                               |
| 04:07:44 | CASH_OUT |  730574.39 |    8 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R4_VIAJE_IMPOSIBLE                                                    |
| 04:07:49 | TRANSFER |   22025.14 |    2 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R2_INCONSISTENCIA_DESTINO,R4_VIAJE_IMPOSIBLE                          |
| 04:07:58 | TRANSFER | 5777706.65 |    8 |   1.0 | BLOQUEAR | R1_VACIADO_CUENTA,R2_INCONSISTENCIA_DESTINO,R3_MONTO_ALTO                               |
+----------+----------+------------+------+-------+----------+-----------------------------------------------------------------------------------------+
+----------+------------+---------------------+---------------------+
| lecturas | terminales |        desde        |        hasta        |
+----------+------------+---------------------+---------------------+
|     6000 |         50 | 2026-11-10 04:07:51 | 2026-11-10 04:17:47 |
+----------+------------+---------------------+---------------------+

# código de salida: 0
```

Figura 36. Decisiones del pipeline, alertas con las reglas que las dispararon y telemetría escrita en Bronze

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud dataflow jobs describe 2026-11-09_20_49_02-5099115295804935858 --region us-central1 --full --format='yaml(id,name,type,
currentState,environment.workerPools[0].machineType,environment.workerPools[0].numWorkers,environment.workerPools[0].autoscaling
Settings,environment.workerPools[0].network,environment.workerPools[0].subnetwork,environment.workerPools[0].ipConfiguration)'
# proyecto: lumina-bank-utec · 2026-11-10 00:09:59 -05

currentState: JOB_STATE_RUNNING
environment:
  workerPools:
  - autoscalingSettings:
      algorithm: AUTOSCALING_ALGORITHM_BASIC
      maxNumWorkers: 1
    ipConfiguration: WORKER_IP_PRIVATE
    machineType: n2-standard-2
    network: default
    numWorkers: 1
    subnetwork: regions/us-central1/subnetworks/lumina-datos-us-central1
id: 2026-11-09_20_49_02-5099115295804935858
name: lumina-fraude-stream-v3
type: JOB_TYPE_STREAMING

# código de salida: 0
```

Figura 37. Job de Dataflow con workers sin IP pública en la subred `lumina-datos-us-central1`

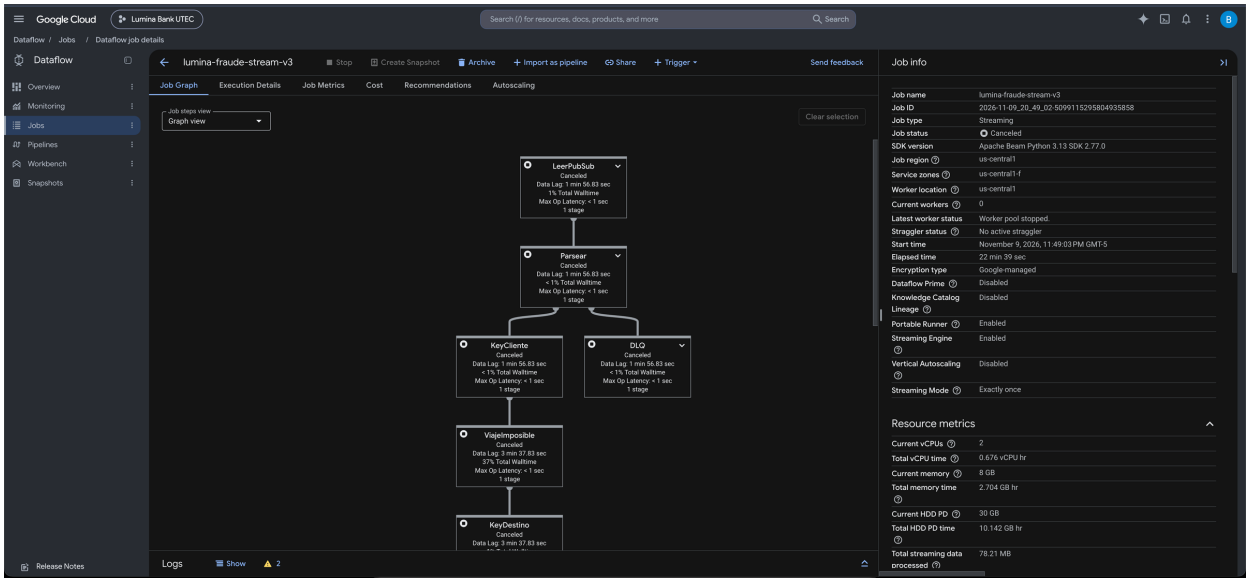


Figura 38. Grafo del job `lumina-fraude-stream-v3`: lectura de Pub/Sub, parseo con salida a DLQ y etapas con estado por cliente. `ViajeImposible` concentra el 37 % del tiempo de procesamiento

6.5 Pruebas de carga

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud dataflow jobs update-options 2026-11-09_20_05_40-16619663328803287800 --region us-central1 --min-num-workers 1 --max-
num-workers 1 --format='yaml(id,name,currentState,runtimeUpdatableParams)'
# proyecto: lumina-bank-utec · 2026-11-09 23:18:50 -05

currentState: JOB_STATE_RUNNING
id: 2026-11-09_20_05_40-16619663328803287800
name: lumina-fraude-stream
runtimeUpdatableParams:
  maxNumWorkers: 1
  minNumWorkers: 1
  workerUtilizationHint: 0.8

# código de salida: 0
```

Figura 39. Ajuste en caliente del rango de workers con `update-options`, el mismo mecanismo que usa el DAG de precalentamiento

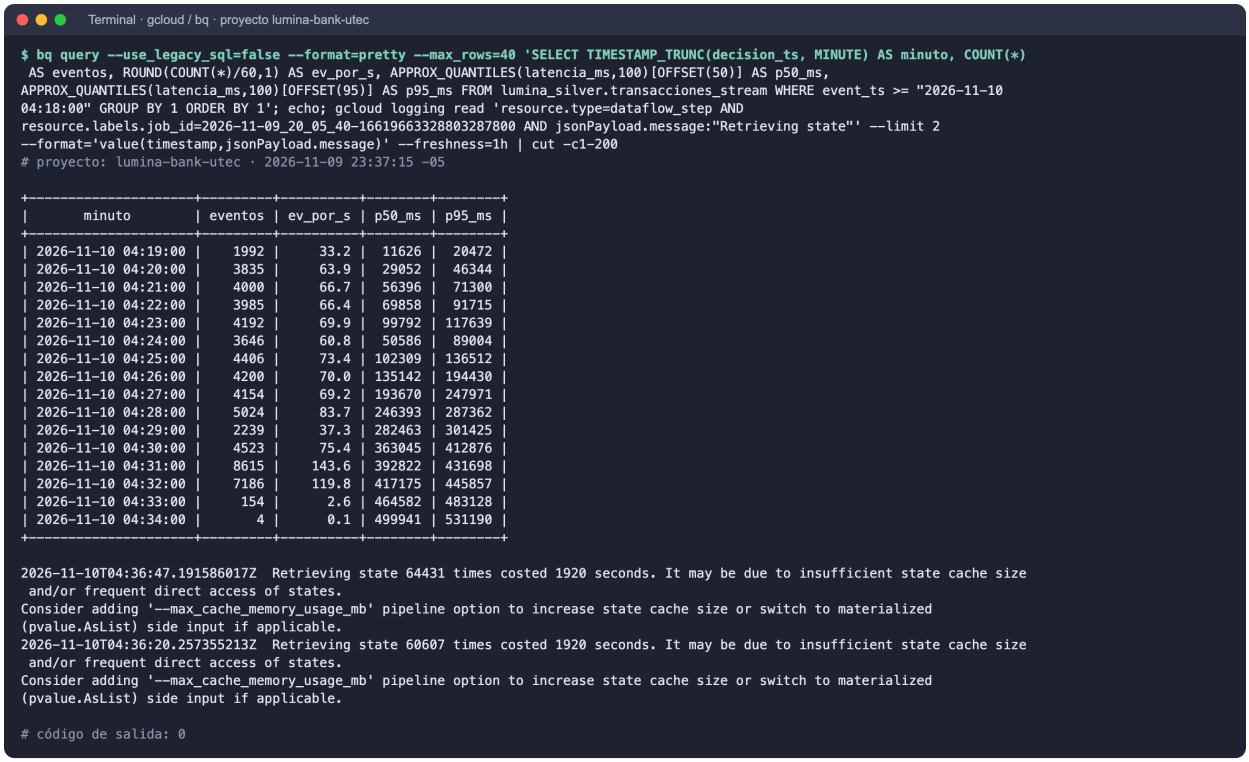


Figura 40. Corrida 1: eventos por minuto y latencia, con el aviso de lecturas de estado lentas

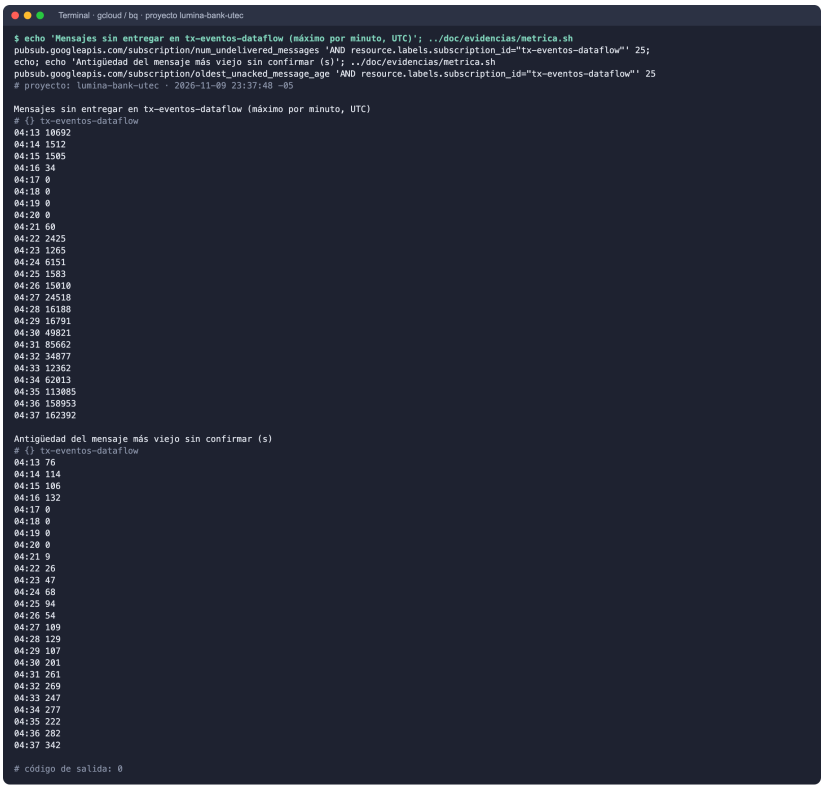


Figura 41. Corrida 1: backlog de la suscripción y antigüedad del mensaje más viejo

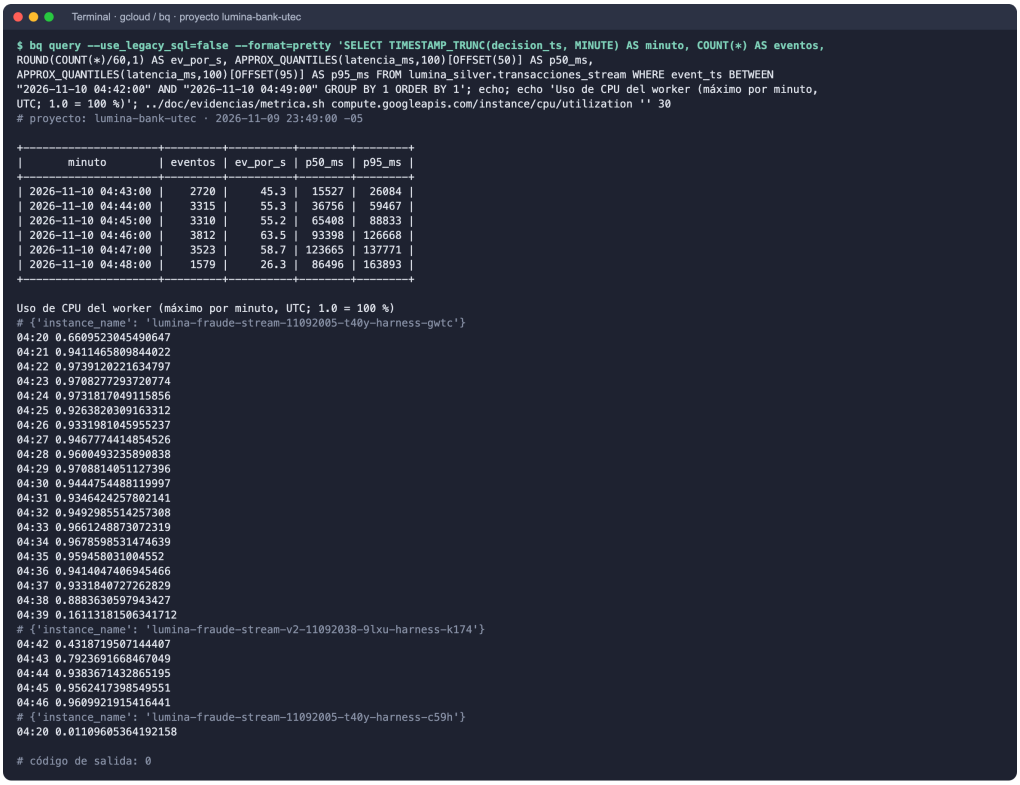


Figura 42. Corrida 2: eventos por minuto y uso de CPU del worker

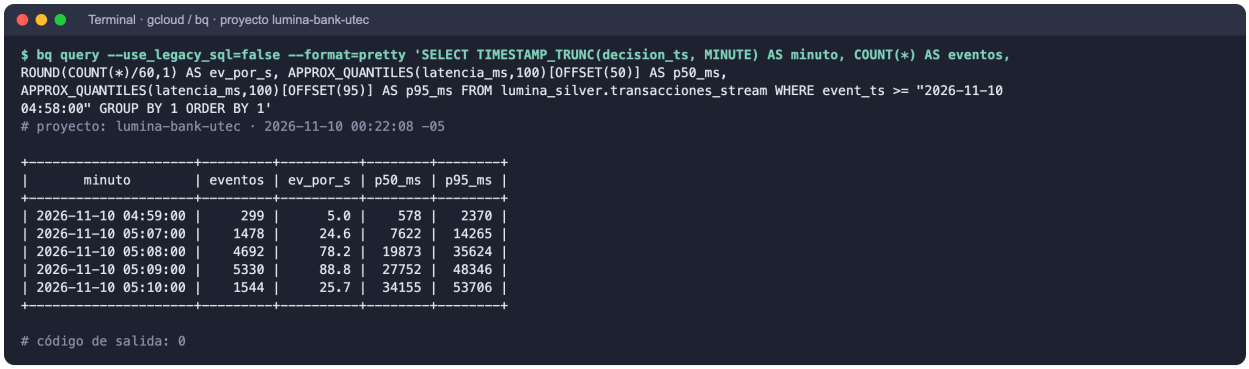


Figura 43. Corrida 3: eventos por minuto con la escritura a BigQuery en lotes. El primer minuto es la prueba de calentamiento a 5 ev/s, con p95 de 2,4 s

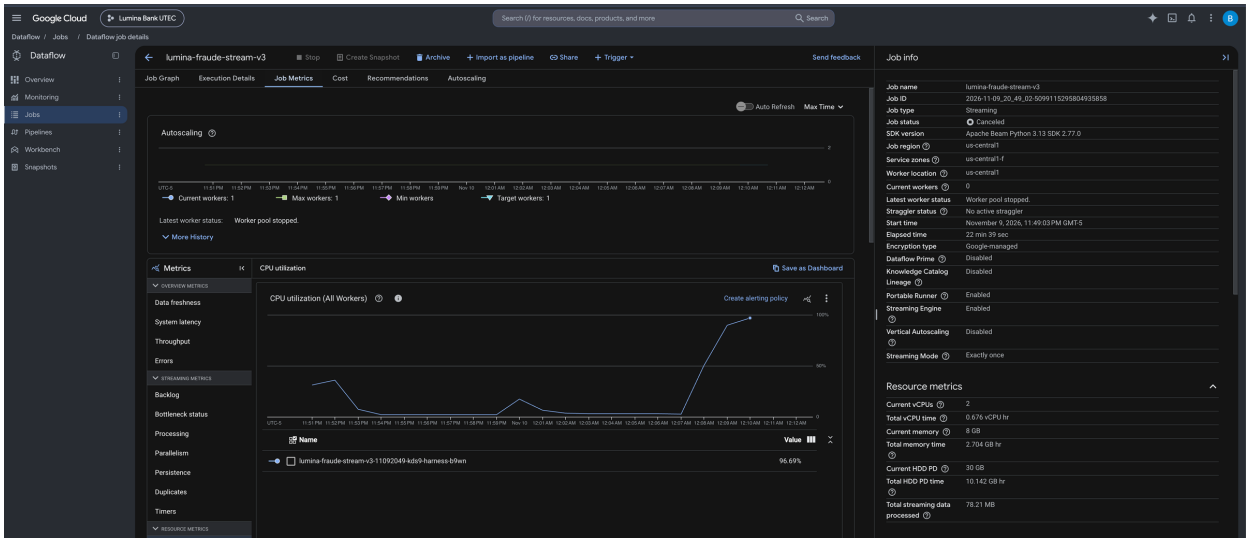


Figura 44. Consola, referencial: métricas del job `v3`: un solo worker (máximo 1) y la CPU sube a 97 % cuando entra la carga

6.6 Gold y BigQuery ML



Figura 45. Creación de los hechos y vistas de la capa Gold

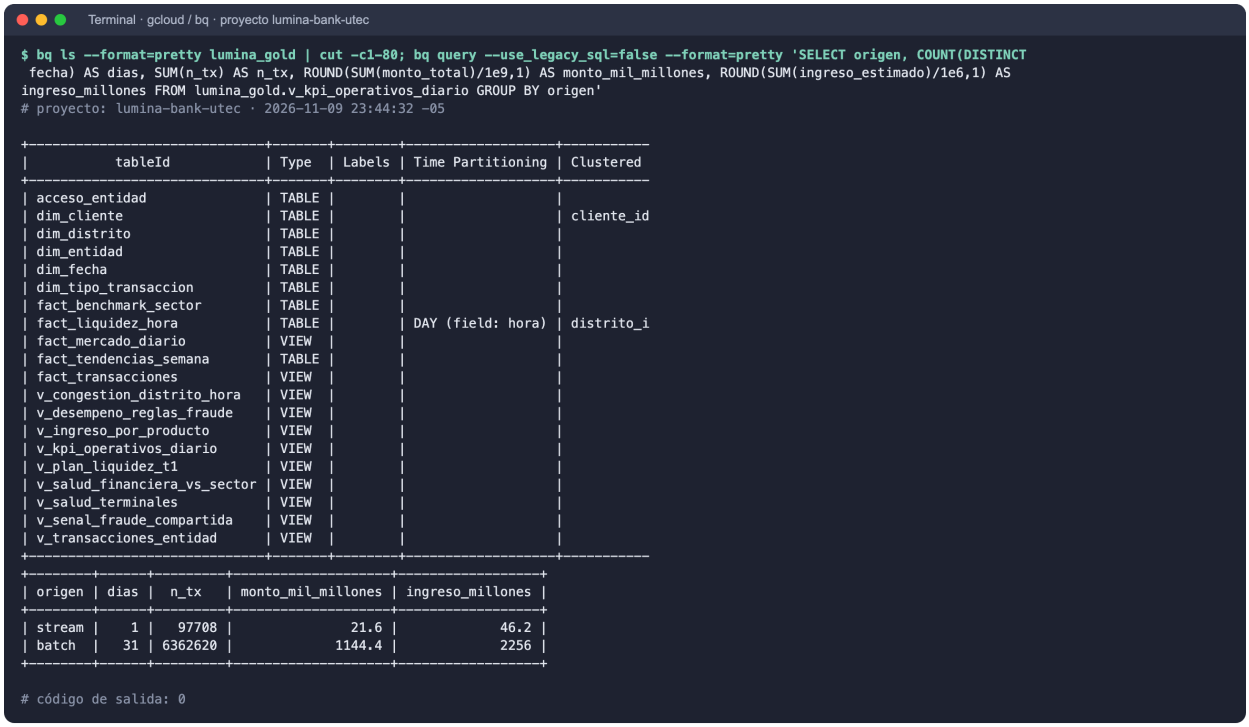


Figura 46. Objetos de Gold y KPI operativos por origen (batch y streaming)

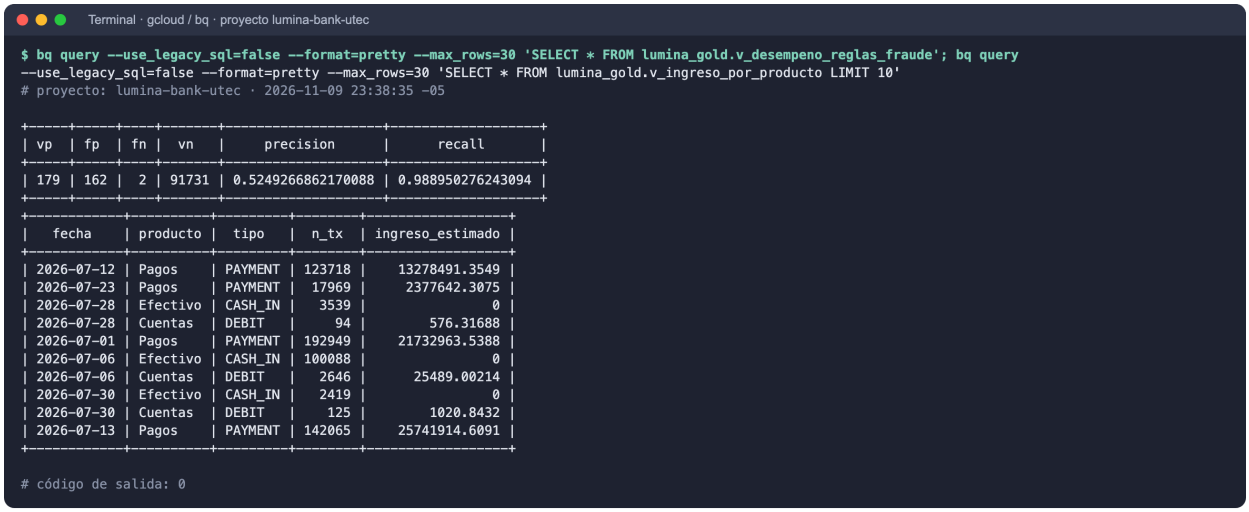


Figura 47. Desempeño de las reglas del pipeline e ingreso por producto

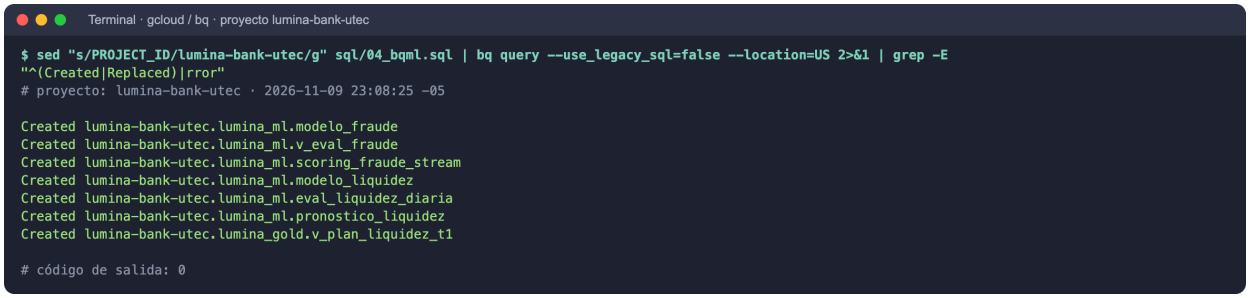


Figura 48. Entrenamiento de los modelos de fraude y liquidez

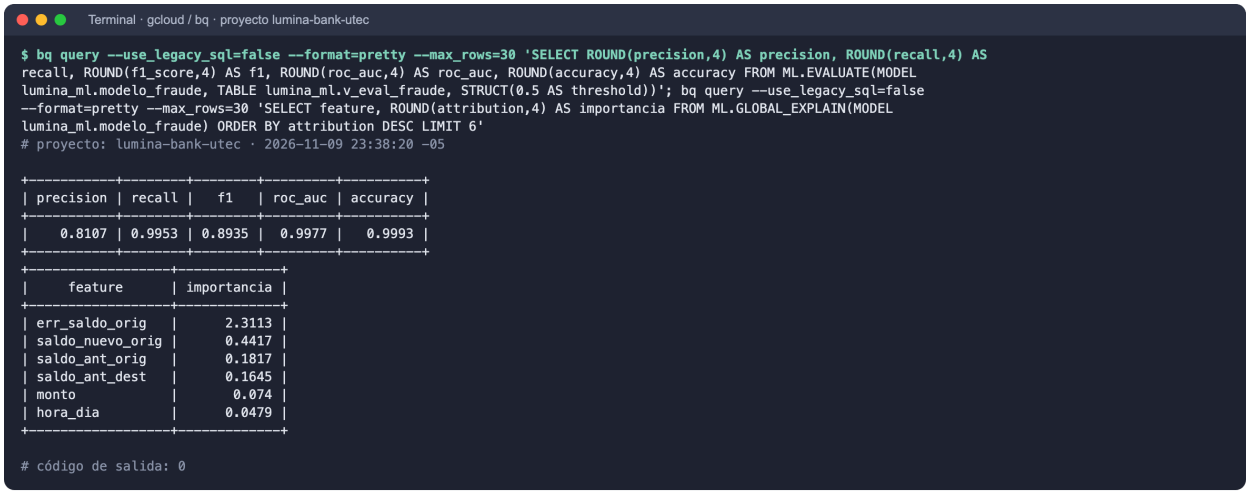


Figura 49. Evaluación del modelo de fraude y variables más importantes

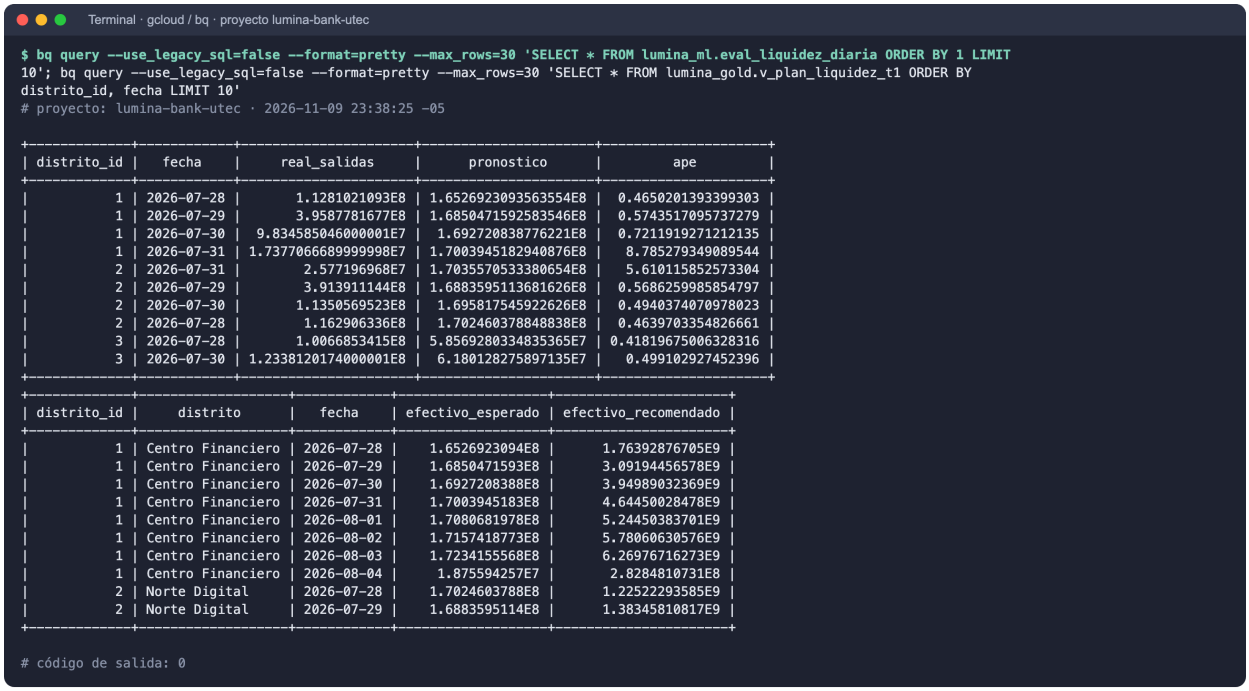


Figura 50. Error del pronóstico por día y plan de liquidez por distrito

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bq query --use_legacy_sql=false --format=pretty 'SELECT distrito_id, ROUND(AVG(ape)*100,1) AS mape_pct, ROUND(AVG(IF(fecha <
"2026-07-31", ape, NULL))*100,1) AS mape_sin_31_jul_pct FROM lumina_ml.eval_liquidez_diaria GROUP BY 1 ORDER BY 1'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:05 -05

+-------------+----------+---------------------+
| distrito_id | mape_pct | mape_sin_31_jul_pct |
+-------------+----------+---------------------+
|           1 |    263.6 |                58.7 |
|           2 |    178.4 |                50.9 |
|           3 |     70.0 |                59.0 |
|           4 |    147.5 |                41.8 |
|           5 |    154.5 |                41.3 |
|           6 |    127.3 |                42.0 |
|           7 |     95.6 |                32.1 |
|           8 |    815.8 |               228.2 |
+-------------+----------+---------------------+

# código de salida: 0
```

Figura 51. MAPE del pronóstico por distrito, con y sin el último día de PaySim

6.7 API de datos en Cloud Run

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud run deploy lumina-api --source api --region us-central1 --service-account sa-lumina-pipelines@lumina-bank-
utec.iam.gserviceaccount.com --set-env-vars PROJECT_ID=lumina-bank-utec --no-allow-unauthenticated --cpu 1 --memory 512Mi
--concurrency 20 --min-instances 0 --max-instances 3 --timeout 60 --quiet
# proyecto: lumina-bank-utec · 2026-11-09 22:53:50 -05

Building using Dockerfile and deploying container to Cloud Run service [lumina-api] in project [lumina-bank-utec] region [us-
central1]
Building and deploying new service...
Validating configuration...done
Creating Container Repository...done
Uploading sources...done
Building Container...done
Creating Revision...done
Routing traffic.....done
Done.
Service [lumina-api] revision [lumina-api-00001-6zb] has been deployed and is serving 100 percent of traffic.
Service URL: https://lumina-api-678938254786.us-central1.run.app
Proxy locally with: gcloud run services proxy lumina-api --region us-central1 --project lumina-bank-utec

# código de salida: 0
```

Figura 52. Despliegue de la API con concurrencia 20 y de 0 a 3 instancias

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud run services describe lumina-api --region us-central1 --format='yaml(metadata.name,status.url,spec.template.metadata.an
notations,spec.template.spec.containerConcurrency,spec.template.spec.timeoutSeconds,spec.template.spec.containers[0].resources.l
imits,spec.template.spec.serviceAccountName)'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:58 -05

metadata:
  name: lumina-api
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/maxScale: '3'
        run.googleapis.com/client-name: gcloud
        run.googleapis.com/client-version: 589.0.0
        run.googleapis.com/startup-cpu-boost: 'true'
    spec:
      containerConcurrency: 20
      containers:
      - resources:
          limits:
            cpu: '1'
            memory: 512Mi
      serviceAccountName: sa-lumina-pipelines@lumina-bank-utec.iam.gserviceaccount.com
      timeoutSeconds: 60
status:
  url: https://lumina-api-gk5x4vgopq-uc.a.run.app

# código de salida: 0
```

Figura 53. Configuración desplegada: máximo de 3 instancias, concurrencia 20, 1 vCPU y 512 MiB

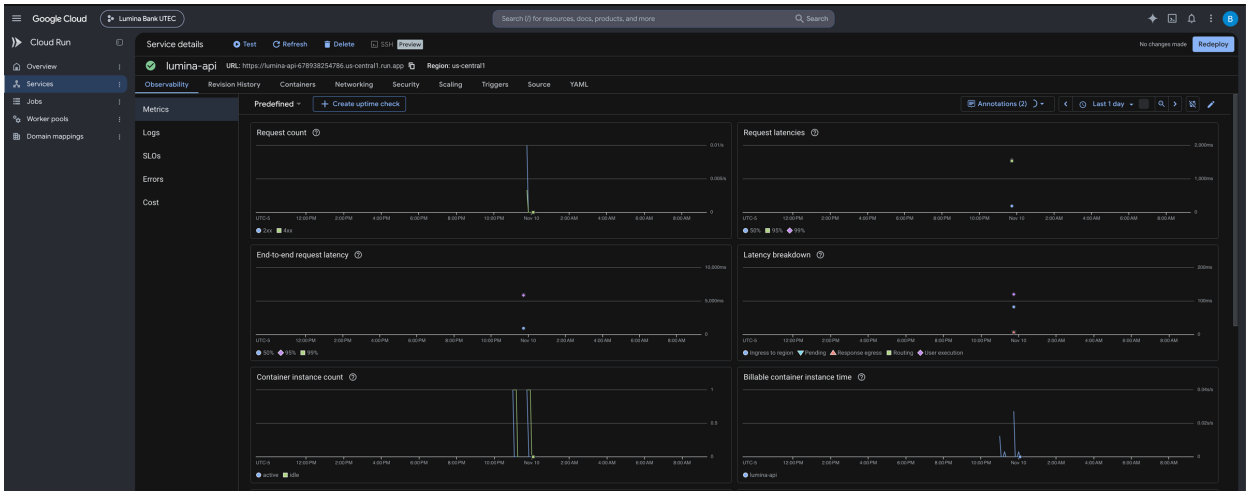


Figura 54. Métricas de `lumina-api` en Cloud Run: solicitudes 2xx y 4xx, latencia e instancias que vuelven a cero cuando no hay tráfico

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ curl -s -o /dev/null -w 'HTTP %{http_code} sin credenciales
' https://lumina-api-678938254786.us-central1.run.app/salud
# proyecto: lumina-bank-utec · 2026-11-09 23:40:00 -05

HTTP 403 sin credenciales

# código de salida: 0
```

Figura 55. Sin credenciales la API responde 403

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ TOKEN=$(gcloud auth print-identity-token); curl -s -H "Authorization: Bearer $TOKEN" https://lumina-api-678938254786.us-
central1.run.app/salud; echo; echo; curl -s -H "Authorization: Bearer $TOKEN" 'https://lumina-api-678938254786.us-
central1.run.app/alertas?minutos=1440&limite=3' | python3 -m json.tool | head -40; echo; curl -s -w '
HTTP %{http_code} · %{time_total}s
' -H "Authorization: Bearer $TOKEN" 'https://lumina-api-678938254786.us-central1.run.app/liquidez/1/pronostico' | head -c 700
# proyecto: lumina-bank-utec · 2026-11-09 23:40:01 -05

{"estado":"ok"}

[
    {
        "event_id": "d9ca8ad4-25fc-4871-99a3-c6878b1c77a3",
        "event_ts": "2026-11-10T04:26:12.593654+00:00",
        "cliente_id": "C1629507595",
        "destino_id": "C88641718",
        "distrito_id": 5,
        "tipo": "TRANSFER",
        "monto": 355244.18,
        "score": 0.5,
        "decision": "REVISAR",
        "reglas": [
            "R3_MONTO_ALTO",
            "R5_CUENTA_MULA"
        ]
    },
    {
        "event_id": "c9e7378b-cdb8-4c5b-9a08-9881a9712e98",
        "event_ts": "2026-11-10T04:26:08.653840+00:00",
        "cliente_id": "C1814584419",
        "destino_id": "C2421516",
        "distrito_id": 4,
        "tipo": "TRANSFER",
        "monto": 4434390.02,
        "score": 0.5,
        "decision": "REVISAR",
        "reglas": [
            "R3_MONTO_ALTO",
            "R5_CUENTA_MULA"
        ]
    },
    {
        "event_id": "39e878b3-0b29-40d2-a23f-10b673d0cf65",

[… 16 líneas más en el log …]
```

Figura 56. Con token, la API devuelve las alertas recientes y el pronóstico de liquidez

7 Pendientes para la Entrega 3

- Conectar Looker Studio a las vistas `v_kpi_*` y armar el tablero de la demo.
- Desplegar los dos DAG en Composer y mostrar una ejecución completa.
- Llevar el modelo de fraude al pipeline (hoy decide con reglas) y desplegar el servicio de autorización.
- Optimizar las etapas con estado del pipeline y repetir la prueba de carga hasta llegar a 500 eventos por segundo por worker.
- Repetir la prueba con autoscaling activo para confirmar que el job escala de 4 a 40 workers sin perder latencia.
- Preparar la demo de punta a punta: simulador, Pub/Sub, Dataflow, alerta, API y tablero.

A Cambios en el repositorio

Archivo	Cambio
`doc/arquitectura/lumina.c4`	Modelo C4 en LikeC4; `render.sh` exporta las vistas
`doc/evidencias/`	Logs de cada comando y generador de capturas
`src/infra/gobierno_seguridad_red.sh`	VPC, firewall, KMS, auditoría, policy tags y Dataplex
`src/sql/05_gobierno_seguridad.sql`	Vistas por entidad y señal de fraude compartida
`src/sql/06_validacion_capacidad.sql`	Análisis de la prueba de carga
`src/orchestration/dag_lumina_precalentamiento.py`	Sube y baja el mínimo de workers en días de pago
`src/sql/02_dimensiones.sql`, `03_gold.sql`	Trends con Colombia como referencia y SEC con el año 2018
`src/infra/setup.sh`	Rol `pubsub.viewer` para la cuenta de los pipelines
`src/dataflow/fraud_streaming_pipeline.py`	Escritura a BigQuery en lotes y parámetros de ejecución medidos